COURSE SLIDES

Certified Kubernetes Administrator (CKA)

WSQ Course Code: TGS-2025054612

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

Minimum 75% attendance required

YOUR TRAINER

About the Trainer

Mohan Pothula

Kubernetes & Cloud-Native Trainer

Role

Principal Kubernetes & DevOps trainer at Tertiary Infotech Academy.

Qualifications

Certified Kubernetes Administrator (CKA) · CKAD · CKS · Docker Certified Associate · AWS Solutions Architect.

Expertise

Kubernetes cluster administration · Docker · CI/CD · kubeadm · etcd · Helm.

Experience

20+ years across DBS Bank, SingTel, Mediacorp and SPH Media.

YOUR TRAINER

About the Trainer

[Trainer Name]

[Title / Role]

Qualifications: _____

Expertise: _____

Experience: _____

Contact / Profile: _____

ICE-BREAKER

Let's Know Each Other

Take a minute to introduce yourself to the class:

Your role

Your name and organisation / role.

Your experience

Any experience with Docker, Kubernetes or Linux administration.

Your goal

Whether you are targeting the CKA exam or production cluster skills.

HOUSE RULES

Ground Rules

Phones on silent

Set your phone to silent mode.

Participate actively

No question is too small — ask freely.

Mutual respect

One conversation at a time.

Be punctual

Return from breaks on time.

Step out quietly

For calls or short breaks.

75% attendance

Required for funding eligibility.

SCHEDULE

Lesson Plan — 3½ Days, 8 hours/day

1 Day 1 · Domain 1

Cluster Architecture, Installation & Configuration

· kubeadm bootstrap, CNI, cluster upgrade (Labs 1–4)
· HA etcd, Helm, Kustomize (Labs 5–7)
· RBAC (Lab 8)

2 Day 2 · Domains 1+2+3

CRDs + Workloads & Scheduling + Services start

· CRDs, CRI/CNI/CSI, Deployments, ConfigMaps (Labs 9–13)
· Secrets, HPA, self-healing, scheduling (Labs 14–16)
· Pod connectivity, Service types (Labs 17–18)

3 Day 3 · Domains 3+4+5

Networking + Storage + Troubleshooting

· Ingress, Gateway API, NetworkPolicy, CoreDNS (Labs 19–22)
· PV/PVC, StorageClass, volume types (Labs 23–25)
· Cluster, node and application troubleshooting (Labs 26–28)

4 Day 4 (½ day) · Domain 5

Monitoring + Service triage + RBAC/scheduling debug

· Monitoring, service networking, RBAC failures (Labs 29–31)
· 12:00 PM lunch · 1:00 PM mock-exam practice
· 3:30 PM tea · 3:45 PM final assessment

Daily timing: 9:00am – 6:00pm · 1-hour lunch · short tea breaks within each day

WHAT YOU'LL ACHIEVE

Learning Outcomes

1. **LO1** Bootstrap a Kubernetes cluster with kubeadm, containerd, and a CNI plugin.
2. **LO2** Perform cluster upgrades, configure HA etcd, and back up/restore cluster state.
3. **LO3** Configure RBAC, Helm, Kustomize, CRDs, and the CNI/CSI/CRI extension interfaces.
4. **LO4** Deploy, scale, schedule, and self-heal workloads with HPA, taints, and affinity.
5. **LO5** Expose services with ClusterIP, NodePort, Ingress, and Gateway API; write NetworkPolicy.
6. **LO6** Provision persistent storage with PV/PVC, StorageClass, and troubleshoot the full cluster stack.

FINAL ASSESSMENT

Assessment

Final Assessment

- Written / MCQ assessment on the LMS
- Mock-exam practice from 1:00 PM; final assessment from 3:45 PM
- Covers all five CKA domains and the hands-on labs
- Open book — slides, Learner Guide & kubernetes.io
- Must be attempted for SSG funding

Funding & Competency

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as Competent

Pass the written / MCQ assessment.

ASSESSMENT RULES

Briefing for Assessment

Clear your space

Keep only approved materials on the table.

No recording

No photos of the assessment.

Work individually

No discussion during the assessment.

Use the LMS

The assessment is delivered online.

Open book

Slides, Learner Guide & kubernetes.io are allowed.

Time's up

Submit when time is up.

01

DAY 1

Cluster Architecture, Installation & Configuration

Domain 1 (25%) — Labs 1–8 · kubeadm · CNI · upgrade · HA · Helm · Kustomize · RBAC

TOPIC 1

Kubernetes Architecture

The control plane, worker nodes and their components

What Makes a Kubernetes Cluster?

- A Kubernetes cluster is a set of nodes: one or more control-plane nodes and worker nodes.
- The control plane stores cluster state in etcd and runs the scheduler and controller loops.
- Worker nodes run the kubelet (node agent), kube-proxy (service routing), container runtime and CNI plugin.
- The kube-apiserver is the single entry point — all tools (kubectl, controllers, kubelets) talk to it.

THE BRAIN OF THE CLUSTER

Control Plane Components

kube-apiserver

- REST gateway to etcd
- AuthN + AuthZ + admission
- All state changes go here

kube-scheduler

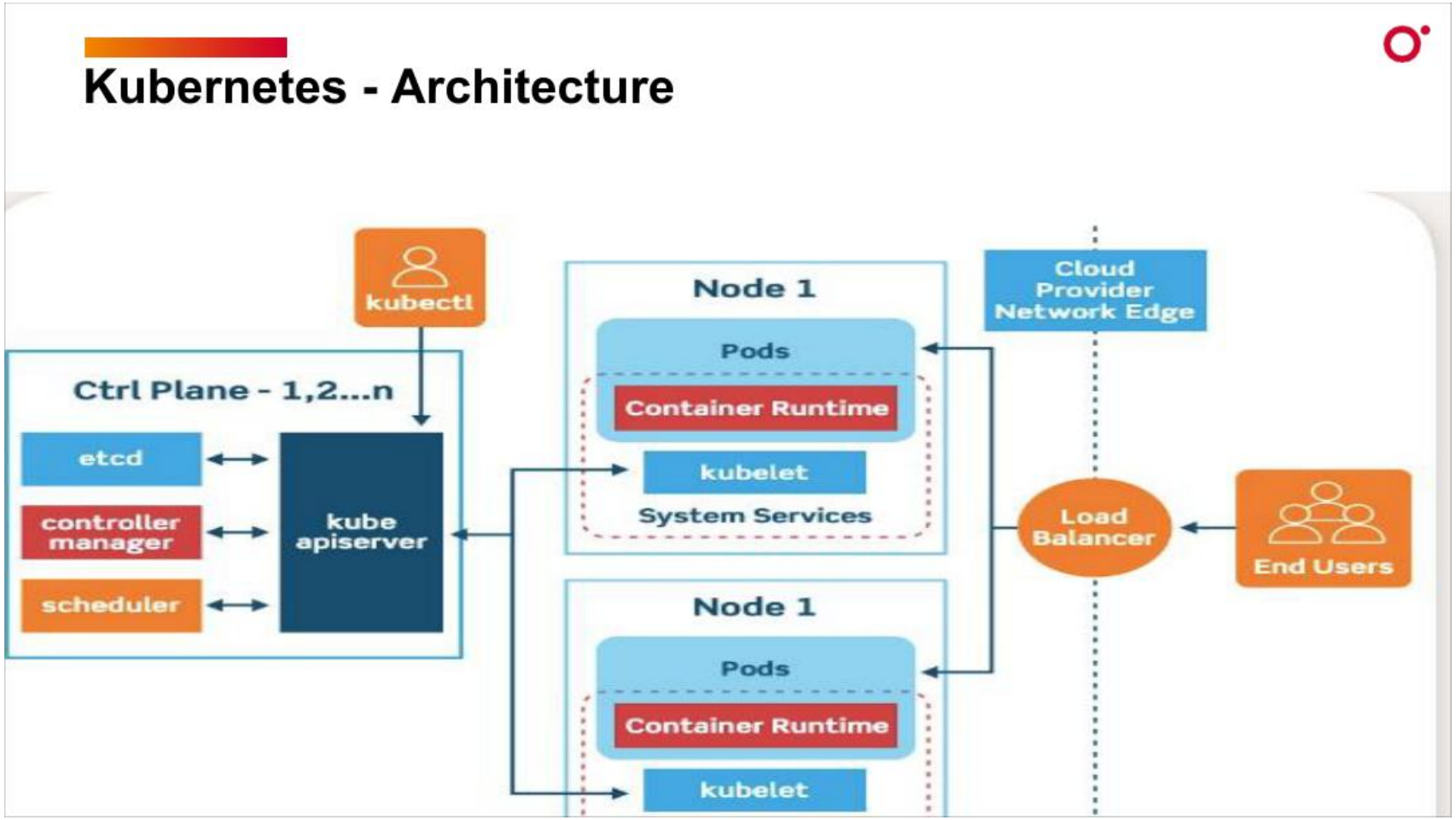
- Selects a node for each pod
- Evaluates resource requests
- taints, affinity, topology

kube-controller-manager

- Reconciliation loops
- Node, ReplicaSet, Job
- Drives actual → desired state

Kubernetes Architecture

Kubernetes Architecture — Control Plane & Worker Nodes



WHERE WORKLOADS RUN

Worker Node Components

- kubelet: the node agent — watches the API server for pods assigned to this node and starts/stops containers.
- kube-proxy: programs iptables or IPVS rules to implement Service ClusterIP and NodePort routing.
- Container runtime: containerd (default) or CRI-O; communicates with kubelet via the CRI gRPC interface.
- CNI plugin: called on every pod create/delete to assign an IP; Calico and Cilium are the most common CKA choices.

TOPIC 2

kubeadm Installation

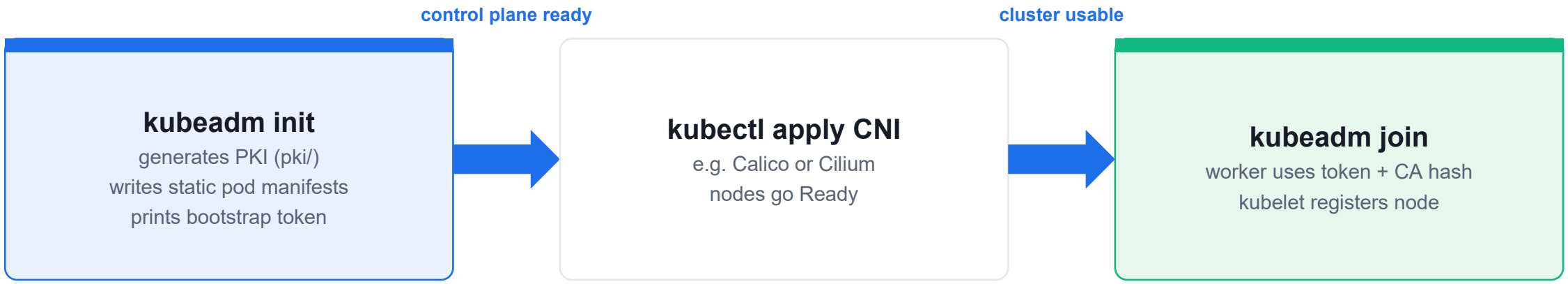
Bootstrap a production-grade cluster in minutes

STEP BY STEP

Install Sequence

- Pre-flight: load overlay + br_netfilter kernel modules; set net.ipv4.ip_forward=1 sysctl; swap off.
- Install containerd with SystemdCgroup=true; then install kubeadm, kubelet and kubectl from pkgs.k8s.io.
- kubeadm init generates the PKI under /etc/kubernetes/pki/, writes static pod manifests and prints the join command.
- Copy /etc/kubernetes/admin.conf to ~/.kube/config; apply a CNI manifest (Calico/Cilium); then kubeadm join on workers.

kubeadm Bootstrap Flow



Manifests: **/etc/kubernetes/manifests/** · PKI: **/etc/kubernetes/pki/** · kubeconfig: ~/.kube/config

kubeadm Prerequisites and Container Runtime

LAB 1 Cluster installation

Prepare two clean Ubuntu nodes for a Kubernetes cluster install: load kernel modules, configure sysctls, install containerd as the CRI, and install kubeadm/kubelet/kubectl from the official pkgs.k8s.io repository. CKA tests this sequence in troubleshooting questions where a node is NotReady due to a misconfigured runtime.

You'll build

Configure kernel modules, sysctls, containerd (SystemdCgroup=true), and install pinned Kubernetes packages on both nodes.

Key commands: cat <<EOF · sudo swapoff · sudo apt · » SystemdCgroup · kubeadm version

LAB 1 · COMMANDS

kubeadm Prerequisites and Container Runtime

Step 1 -- Load kernel modules

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter
```

Step 2 -- Set required sysctls

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system
```

LAB 1 · COMMANDS

kubeadm Prerequisites and Container Runtime (cont.)

Step 3 -- Disable swap

```
sudo swapoff -a
sudo sed -i '/ swap / s/^(.*)$/#\1/g' /etc/fstab
```

Step 4 -- Install containerd

```
sudo apt update && sudo apt install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd && sudo systemctl enable containerd
```

Note SystemdCgroup = true aligns containerd's cgroup driver with kubelet. A mismatch is the #1 cause of 'node NotReady' in fresh clusters.

LAB 1 · COMMANDS

kubeadm Prerequisites and Container Runtime (cont.)

Step 5 -- Install kubeadm, kubelet, kubectl

```
sudo apt install -y apt-transport-https ca-certificates curl gpg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.31/deb/ /' | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt update && sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Step 6 -- Verify

```
kubeadm version
kubectl version --client
sudo systemctl status containerd --no-pager | head
```

LAB 1 · TEST IT

kubeadm Prerequisites and Container Runtime

Test it

kubeadm version shows v1.31.x, containerd is active, and crictl reports the runtime version.

Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

kubeadm Bootstrap: init, kubeconfig, Worker Join, Cluster PKI

LAB 2

Cluster installation

Run kubeadm init to create the control plane, configure kubectl access via kubeconfig, join a worker node using the bootstrap token, explore the PKI certificates under /etc/kubernetes/pki/, take an etcd snapshot, and inspect static pod manifests. The CKA exam always includes an etcd backup/restore question.

You'll build

Run kubeadm init, copy admin.conf, join worker, explore PKI, take etcd snapshot with etcdctl.

Key commands: kubeadm init · mkdir -p · ls /etc/kubernetes/manifests/ · ls /etc/kubernetes/pki/ · export ETCDCTL_API=3 · » Always · kubeadm toke

LAB 2 · COMMANDS

kubeadm Bootstrap: init, kubeconfig, Worker Join, Cluster PKI

Step 1 -- Initialize the control plane

```
kubeadm init \
  --kubernetes-version=v1.35.0 \
  --pod-network-cidr=192.168.0.0/16 \
  --service-cidr=10.96.0.0/12 \
  --apiserver-advertise-address=$(hostname -I | awk '{print $1}')
```

Step 2 -- Configure kubectl

```
mkdir -p $HOME/.kube
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes
```

Step 3 -- Explore static pod manifests

```
ls /etc/kubernetes/manifests/
grep 'image:' /etc/kubernetes/manifests/kube-apiserver.yaml
```

LAB 2 · COMMANDS

kubeadm Bootstrap: init, kubeconfig, Worker Join, Cluster PKI (cont.)

Step 4 -- Explore cluster PKI

```
ls /etc/kubernetes/pki/
kubeadm certs check-expiration
openssl x509 -in /etc/kubernetes/pki/ca.crt -noout -dates
```

Step 5 -- Take an etcd snapshot

```
export ETCDCTL_API=3
export ETCDCTL_ENDPOINTS=https://127.0.0.1:2379
export ETCDCTL_CACERT=/etc/kubernetes/pki/etcd/ca.crt
export ETCDCTL_CERT=/etc/kubernetes/pki/etcd/server.crt
export ETCDCTL_KEY=/etc/kubernetes/pki/etcd/server.key
etcdctl snapshot save /opt/etcd-backup-$(date +%Y%m%d).db
etcdctl snapshot status /opt/etcd-backup-$(date +%Y%m%d).db --write-out=table
```

Note Always set ETCDCTL_API=3. etcd backup/restore is guaranteed to appear on the CKA exam.

LAB 2 · COMMANDS

kubeadm Bootstrap: init, kubeconfig, Worker Join, Cluster PKI (cont.)

Step 6 -- Join worker node

```
# On control plane -- generate join command
kubeadm token create --print-join-command
# Run the output on WORKER NODE
# Back on control plane:
kubectl get nodes -o wide
```

LAB 2 · TEST IT

kubeadm Bootstrap: init, kubeconfig, Worker Join, Cluster PKI

Test it

kubectl get nodes shows both nodes (NotReady until CNI installed). etcd snapshot file exists at /opt/.

Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Tea Break

15 minutes

TOPIC 3

CNI — Container Network Interface

Pod networking with Calico and Cilium

How Pods Get IPs

- CNI is called by the kubelet on every pod creation — the plugin assigns an IP from the pod CIDR.
- Calico uses BGP or VXLAN; Cilium uses eBPF for high-performance networking and advanced observability.
- CNI binaries live in /opt/cni/bin/; configuration files live in /etc/cni/net.d/.
- Without a CNI plugin nodes stay NotReady — applying the CNI manifest is a mandatory post-init step.

PICK YOUR CNI

CNI Plugin Choices

Calico

- BGP or VXLAN
- NetworkPolicy support
- Most popular CKA choice

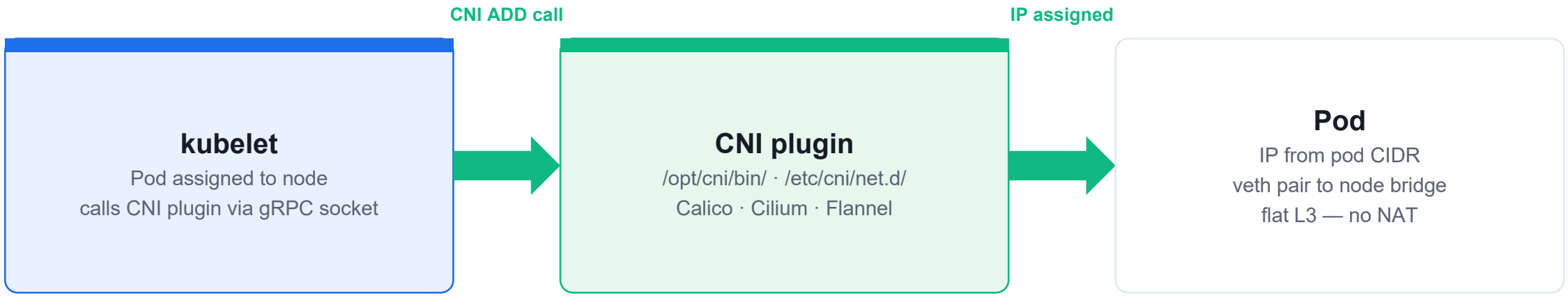
Cilium

- eBPF dataplane
- Advanced observability
- Kubernetes 1.21+ native

Flannel

- VXLAN overlay
- Simple, no NetworkPolicy
- Development / lab only

CNI — How Pods Get Their IP



Without a CNI manifest the node stays **NotReady** — applying it is mandatory after kubeadm init.

DAY 1 · CNI

Install Calico CNI and Verify Cross-Node Pod Ping

LAB 3

CNI networking

Install Calico as the CNI plugin using the Tigera Operator pattern, watch nodes transition to Ready, deploy test pods on different nodes, and validate cross-node pod-to-pod communication. The pod CIDR in Calico must match the --pod-network-cidr from kubeadm init.

You'll build

Apply tigera-operator.yaml + custom-resources.yaml, watch nodes go Ready, ping across nodes.

Key commands: kubectl get · curl -O · kubectl run · » Kubernetes

LAB 3 · COMMANDS

Install Calico CNI and Verify Cross-Node Pod Ping

Step 1 -- Confirm nodes are NotReady before CNI

```
kubectl get nodes -o wide
kubectl get pods -n kube-system
```

Step 2 -- Install Calico Operator

```
curl -O https://raw.githubusercontent.com/projectcalico/calico/v3.29.0/manifests/tigera-operator.yaml
curl -O https://raw.githubusercontent.com/projectcalico/calico/v3.29.0/manifests/custom-resources.yaml
kubectl apply -f tigera-operator.yaml
kubectl apply -f custom-resources.yaml
```

Step 3 -- Watch nodes transition to Ready

```
kubectl get nodes -w
kubectl get pods -n calico-system
```

LAB 3 · COMMANDS

Install Calico CNI and Verify Cross-Node Pod Ping (cont.)

Step 4 -- Test cross-node pod ping

```
kubectl run pod-cp --image=nicolaka/netshoot -- sleep infinity
kubectl run pod-worker --image=nicolaka/netshoot -- sleep infinity
kubectl get pods -o wide
POD_WORKER_IP=$(kubectl get pod pod-worker -o jsonpath='{.status.podIP}')
kubectl exec pod-cp -- ping -c 4 $POD_WORKER_IP
```

Note Kubernetes requires a flat pod network -- cross-node pod-to-pod communication without NAT. A failed ping indicates CNI misconfiguration.

LAB 3 · TEST IT

Install Calico CNI and Verify Cross-Node Pod Ping

Test it

Both nodes are Ready. Cross-node ping returns 0% packet loss. coredns pods are Running.

Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

TOPIC 4

Cluster Upgrade

Upgrade control plane then workers with kubeadm

Upgrade Strategy

- Upgrades are one minor version at a time — you cannot skip from 1.29 to 1.31 directly.
- Always upgrade the control plane first; then drain, upgrade and uncordon each worker node.
- Use apt-mark unhold / hold around the install step to manage package pins.
- kubeadm upgrade apply for the control plane; kubeadm upgrade node for each worker.

Cluster Upgrade — Step by Step

drain the node
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data

→ Cordons the node (no new pods) and evicts existing pods safely before upgrade.

upgrade kubeadm
apt-mark unhold kubeadm && apt install kubeadm=v1.31.0-1.1

→ Run kubeadm upgrade apply v1.31.0 on the control plane; kubeadm upgrade node on workers.

upgrade kubelet + kubectl
apt install kubelet=v1.31.0-1.1 kubectl=v1.31.0-1.1 && systemctl restart kubelet

→ Verify: kubectl get nodes — the node shows the new version after kubelet restart.

uncordon
kubectl uncordon <node>

→ Restores scheduling. Repeat drain → upgrade → uncordon for each worker node in turn.

Upgrade **control plane first**, then each worker. apt-mark hold/unhold kubeadm kubelet kubectl around each step.

Cluster Upgrade: v1.34 to v1.35 with kubeadm drain/uncordon

LAB 4

Cluster upgrade

Upgrade a two-node cluster from v1.34 to v1.35 following the official kubeadm upgrade procedure. The exact sequence: unhold, install kubeadm, upgrade plan, upgrade apply, drain, upgrade kubelet/kubectl, uncordon. Missing any step is a common exam mistake.

You'll build

Upgrade control plane with kubeadm upgrade apply, drain node, upgrade kubelet, uncordon. Repeat for worker.

Key commands: curl -fsSL · apt-mark unhold · kubeadm upgrade · kubectl drain · » The

LAB 4 · COMMANDS

Cluster Upgrade: v1.34 to v1.35 with kubeadm drain/uncordon

Step 1 -- Add v1.35 apt repository

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.35/deb/Release.key | \
  gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
apt-get update
```

Step 2 -- Upgrade kubeadm on control plane

```
apt-mark unhold kubeadm
apt-get install -y kubeadm=1.35.0-*
apt-mark hold kubeadm
kubeadm version
```

Step 3 -- Run upgrade plan and apply

```
kubeadm upgrade plan
kubeadm upgrade apply v1.35.0
```

LAB 4 · COMMANDS

Cluster Upgrade: v1.34 to v1.35 with kubeadm drain/uncordon (cont.)

Step 4 -- Drain, upgrade kubelet, uncordon control plane

```
kubectl drain controlplane --ignore-daemonsets --delete-emptydir-data
apt-mark unhold kubelet kubectl
apt-get install -y kubelet=1.35.0-* kubectl=1.35.0-*
apt-mark hold kubelet kubectl
systemctl daemon-reload && systemctl restart kubelet
kubectl uncordon controlplane
```

LAB 4 · COMMANDS

Cluster Upgrade: v1.34 to v1.35 with kubeadm drain/uncordon (cont.)

Step 5 -- Upgrade worker node

```
# On WORKER NODE:
apt-mark unhold kubeadm && apt-get install -y kubeadm=1.35.0-*
kubeadm upgrade node
# On CONTROL PLANE:
kubectl drain worker01 --ignore-daemonsets --delete-emptydir-data
# Back on WORKER:
apt-mark unhold kubelet kubectl
apt-get install -y kubelet=1.35.0-* kubectl=1.35.0-*
systemctl daemon-reload && systemctl restart kubelet
# On CONTROL PLANE:
kubectl uncordon worker01 && kubectl get nodes
```

Note The VERSION column in 'kubectl get nodes' shows kubelet version, not API server version. systemctl daemon-reload is required after binary update before restarting kubelet.

LAB 4 · TEST IT

Cluster Upgrade: v1.34 to v1.35 with kubeadm drain/uncordon

Test it

kubectl get nodes shows both nodes as Ready at v1.35.0. kubectl version shows Server v1.35.0.

Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Lunch Break

1 hour

TOPIC 5

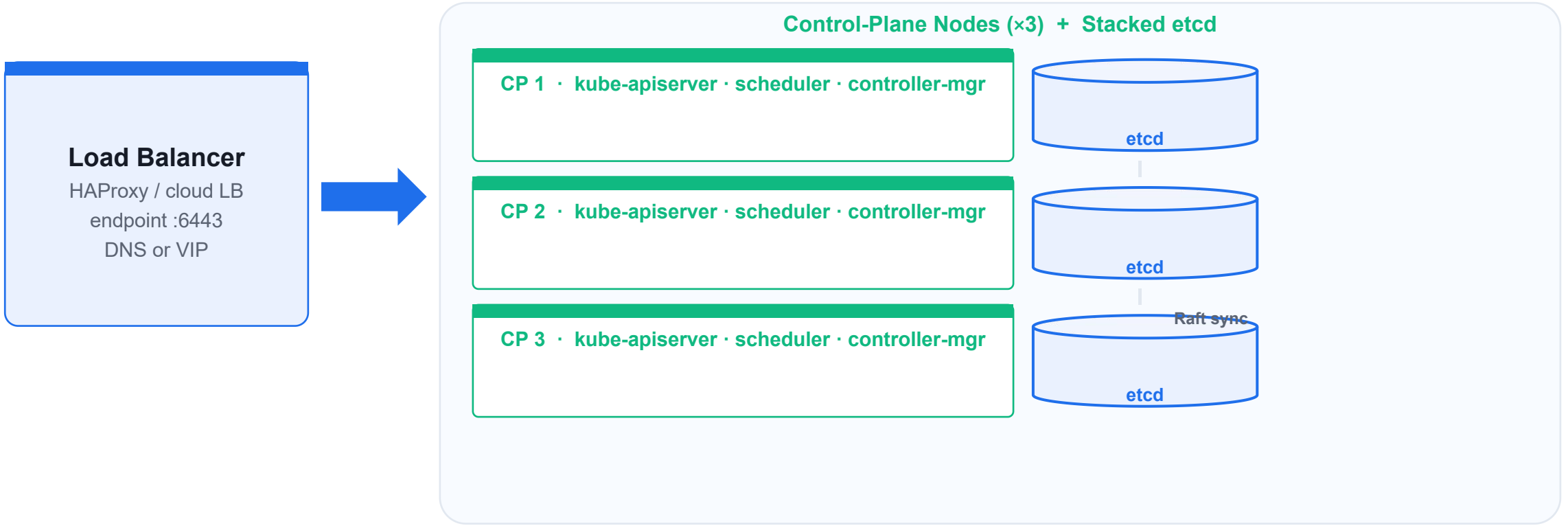
High Availability — etcd

Stacked etcd topology and backup/restore

HA Cluster Design

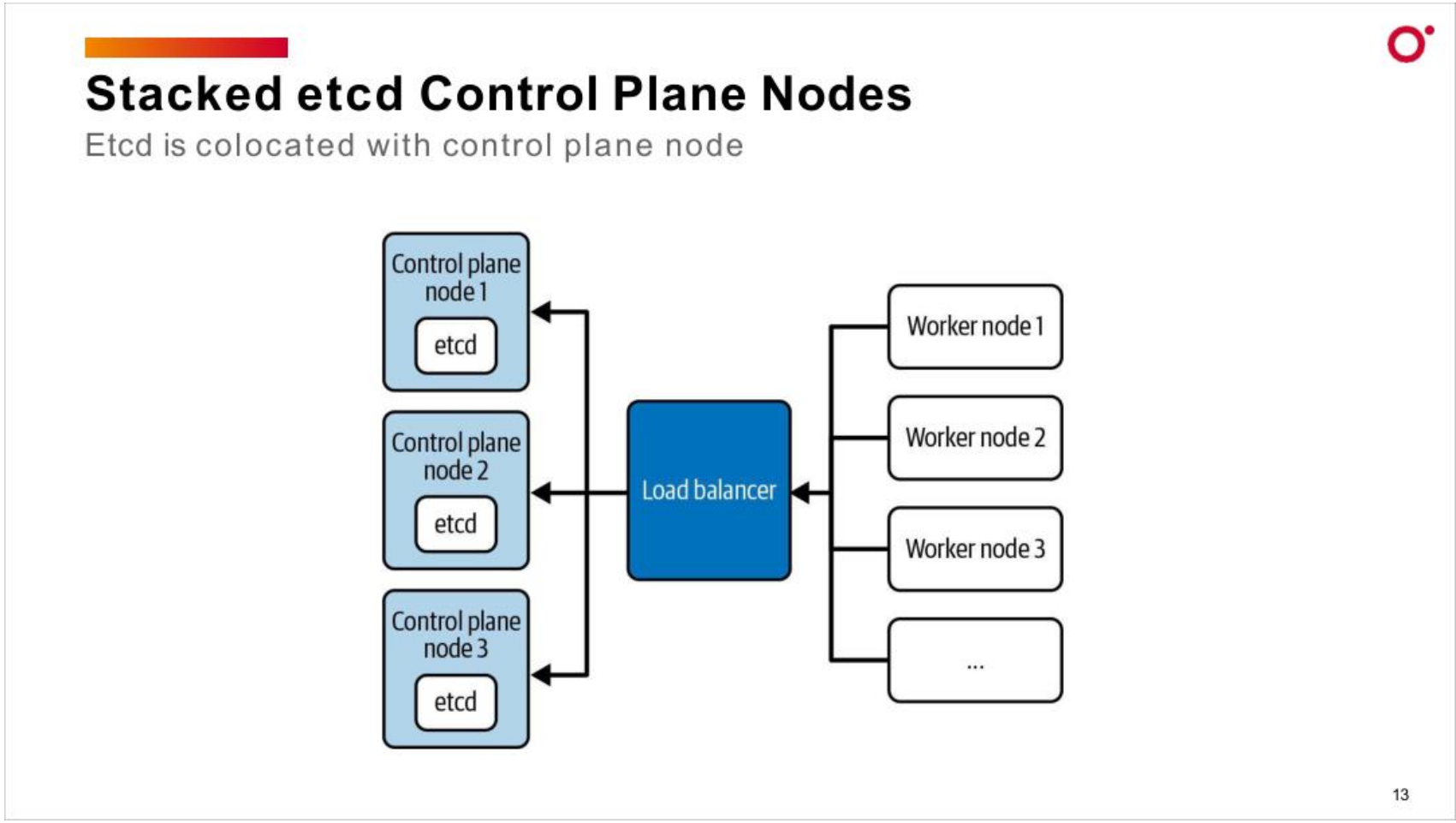
- Stacked etcd: each control-plane node runs its own etcd member — simplest HA topology.
- External etcd: separate etcd cluster decoupled from control-plane nodes — more resilient but more infra.
- All CP nodes must point to the same load-balancer endpoint (--control-plane-endpoint).
- Quorum: (N-1)/2 failures tolerated — 3 nodes survive 1; 5 nodes survive 2. Always use odd numbers.

HA Stacked etcd Topology

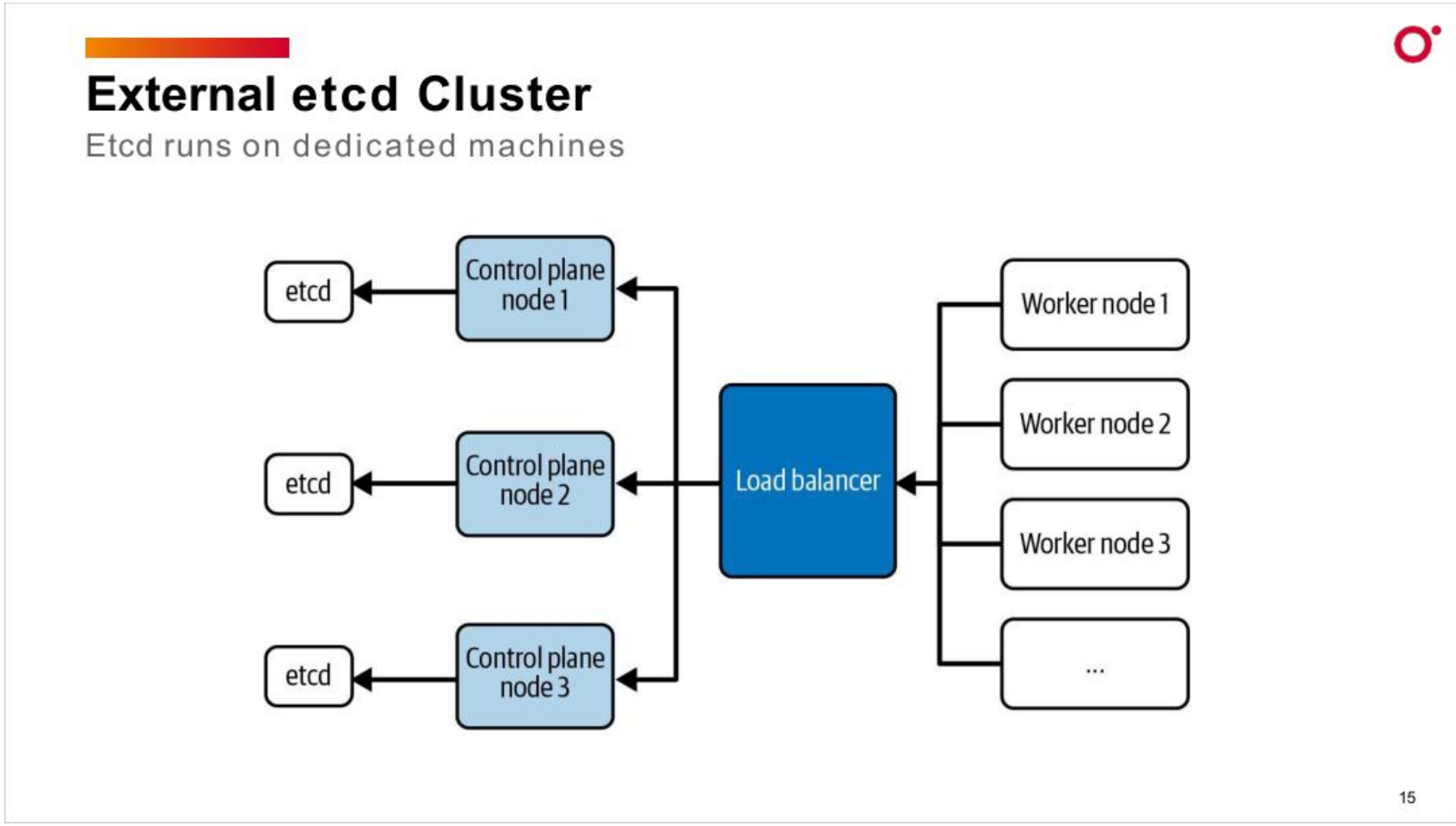


Stacked etcd: **each CP node runs its own etcd member — state replicated via Raft**. 3 CPs tolerate 1 failure (quorum = 2).

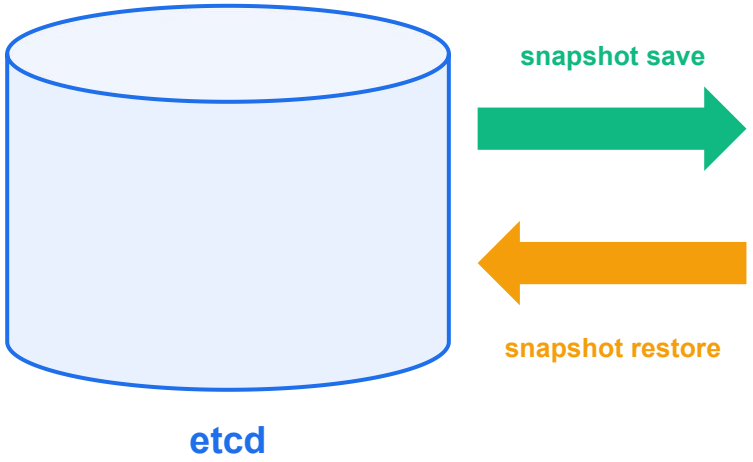
Stacked etcd — HA Control Plane Topology



External etcd Cluster — Dedicated Topology



etcd Backup and Restore



snapshot.db

/backup/etcd.db

etcdctl snapshot status

snapshot restore --data-dir ...

Required etcdctl flags:

--endpoints=https://127.0.0.1:2379
--cacert /etc/kubernetes/pki/etcd/ca.crt
--cert /etc/kubernetes/pki/etcd/server.crt
--key /etc/kubernetes/pki/etcd/server.key

After restore: **update etcd static pod --data-dir** → kubelet auto-restarts etcd. Set **ETCDCTL_API=3** before all etcdctl commands.

HA Control Plane: HAProxy + Keepalived VIP and --upload-certs

LAB 5

High availability

Set up a highly available control plane topology using HAProxy as a TCP load balancer and Keepalived to provide a floating VIP. Initialize the first control plane with --control-plane-endpoint pointing to the VIP and --upload-certs to share certs. CKA tests etcd quorum and cluster resilience questions.

You'll build

Configure HAProxy and Keepalived, run kubeadm init with --control-plane-endpoint and --upload-certs, join additional control plane nodes, verify etcd member list.

Key commands: apt-get install · kubeadm init · » --control-plane-endpoint · kubeadm join · kubectl -n

LAB 5 · COMMANDS

HA Control Plane: HAProxy + Keepalived VIP and --upload-certs

Step 1 -- Install and configure HAProxy

```
apt-get install -y haproxy
systemctl restart haproxy && systemctl enable haproxy
```

Step 2 -- Configure Keepalived with VIP

```
apt-get install -y keepalived
# MASTER: state MASTER, priority 101
# BACKUP: state BACKUP, priority 100
systemctl restart keepalived && systemctl enable keepalived
```

Step 3 -- Initialize first control plane with VIP

```
kubeadm init \
  --kubernetes-version=v1.35.0 \
  --control-plane-endpoint='192.168.1.100:6443' \
  --pod-network-cidr=192.168.0.0/16 \
  --upload-certs
mkdir -p $HOME/.kube && cp /etc/kubernetes/admin.conf $HOME/.kube/config
```

LAB 5 · COMMANDS

HA Control Plane: HAProxy + Keepalived VIP and --upload-certs (cont.)

Note --control-plane-endpoint must point to the VIP address and is baked into all certs/kubeconfigs. --upload-certs stores encrypted certs in a Secret. The --certificate-key expires after 2 hours.

Step 4 -- Join additional control plane nodes

```
kubeadm join 192.168.1.100:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>
```

Step 5 -- Verify etcd HA cluster

```
kubectl -n kube-system exec -it etcd-control-plane-01 -- \
  etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key member list
```

LAB 5 · TEST IT

HA Control Plane: HAProxy + Keepalived VIP and --upload-certs

Test it

etcdctl member list shows all control plane nodes as healthy. kubectl get nodes shows 3+ nodes. VIP migrates to secondary when primary API server is stopped.

Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

TOPIC 6

Helm & Kustomize

Package management and configuration overlays

HELM

Helm — The Package Manager

- A Helm chart is a bundle of templated YAML; a release is one deployed instance of a chart.
- Workflow: helm repo add → helm install → helm upgrade → helm rollback → helm uninstall.
- --set key=value overrides chart values at install/upgrade time.
- helm history <release> lists every revision; helm get manifest shows what was applied.

EXAM-TESTED COMMANDS

Helm Commands to Know

Install & Upgrade

- helm repo add <name> <url>
- helm install <rel> <chart>
- helm upgrade <rel> <chart> --set

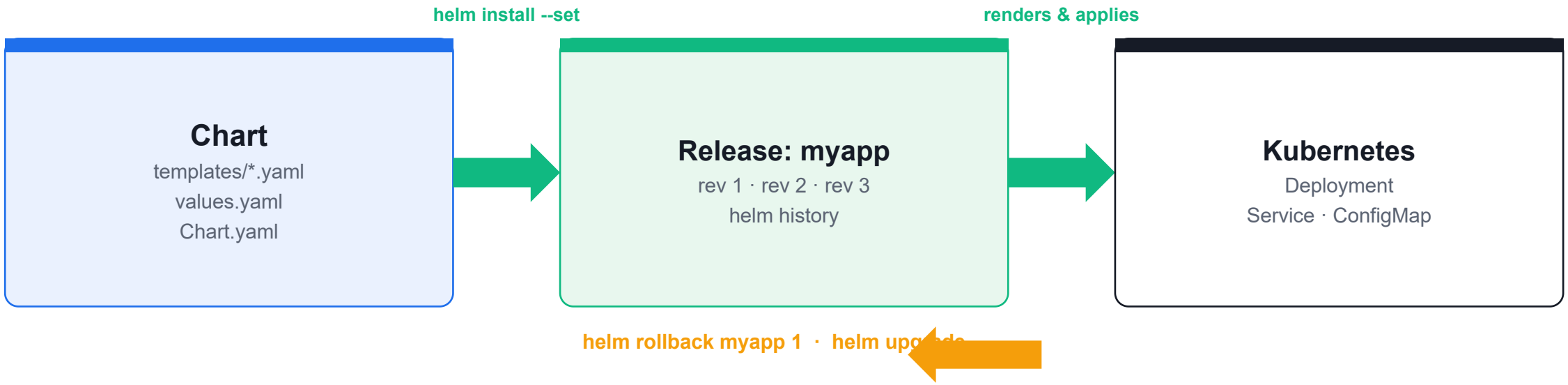
Inspect & Debug

- helm list / helm history
- helm get manifest <rel>
- helm get values <rel>

Rollback & Remove

- helm rollback <rel> <rev>
- helm uninstall <rel>
- --dry-run for testing

Helm — Chart, Release, Revisions



install → upgrade → rollback → uninstall. --set overrides values; helm history shows every revision.

DAY 1 · HELM

Helm: Install, Upgrade, Rollback, --set, -f values.yaml, helm template

LAB 6

Helm package manager

Master the complete Helm workflow: adding repos, installing charts, overriding values with --set and -f, upgrading releases, rollbacks, and using helm template to render manifests. Helm is explicitly allowed as a reference during the CKA exam at helm.sh/docs.

You'll build

Add bitnami repo, install nginx chart with custom values, upgrade, rollback, use helm template for GitOps-style rendering.

Key commands: curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 · helm install · cat <<'EOF' · helm rollback · helm templ

LAB 6 · COMMANDS

Helm: Install, Upgrade, Rollback, --set, -f values.yaml, helm template

Step 1 -- Install Helm and add repos

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
```

Step 2 -- Install a chart with value overrides

```
helm install my-nginx bitnami/nginx \
  --namespace web --create-namespace \
  --set service.type=NodePort \
  --set replicaCount=2
helm list -n web
helm status my-nginx -n web
```

LAB 6 · COMMANDS

Helm: Install, Upgrade, Rollback, --set, -f values.yaml, helm template (cont.)

Step 3 -- Upgrade with -f values.yaml

```
cat <<'EOF' > my-values.yaml
replicaCount: 3
service:
  type: ClusterIP
EOF
helm upgrade my-nginx bitnami/nginx -n web -f my-values.yaml
helm history my-nginx -n web
```

Step 4 -- Roll back a release

```
helm rollback my-nginx -n web
helm history my-nginx -n web
```

Step 5 -- Render manifests without installing

```
helm template my-nginx bitnami/nginx --set replicaCount=2
helm template my-nginx bitnami/nginx -f my-values.yaml > rendered.yaml
```

LAB 6 · COMMANDS

Helm: Install, Upgrade, Rollback, --set, -f values.yaml, helm template (cont.)

Note helm template is used in GitOps workflows. -f values override defaults; --set overrides -f when both are specified.

LAB 6 · TEST IT

Helm: Install, Upgrade, Rollback, --set, -f values.yaml, helm template

Test it

helm list -n web shows my-nginx deployed. helm history shows at least 2 revisions. helm rollback reverts to the previous version.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

KUSTOMIZE

Kustomize — Overlays Without Templates

- Kustomize reuses a base set of manifests and applies environment-specific patches on top.
- namePrefix and commonLabels modify every resource in an overlay without touching the base.
- The images block overrides image tags; strategic-merge patches fine-tune individual fields.
- Built into kubectl: kubectl apply -k and kubectl kustomize — no extra binary required.

BUILD vs APPLY

kubectl apply -k vs kubectl kustomize

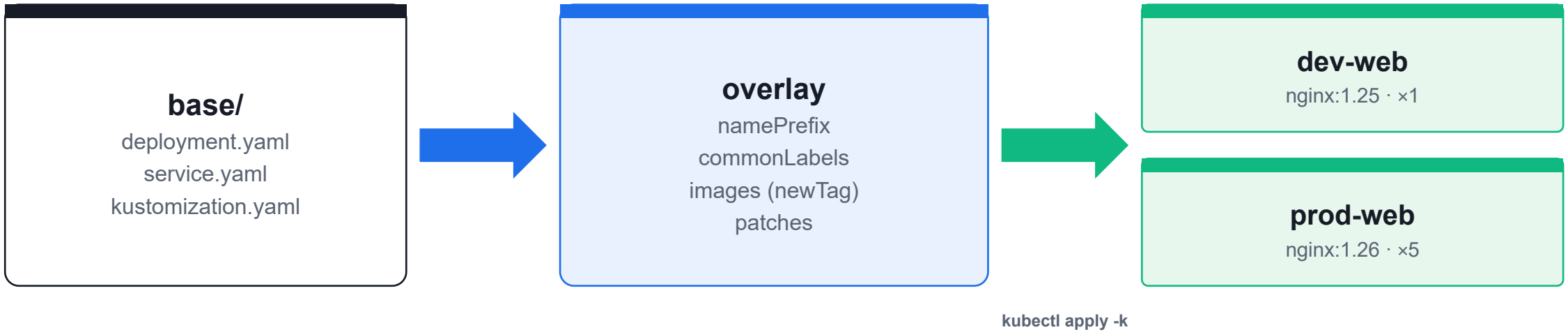
apply -k

- kubectl apply -k ./overlay/
- Builds the manifest AND applies it in one step
- Use for deploying to the cluster
- Equivalent to: kustomize build | kubectl apply -f -

kustomize (print only)

- kubectl kustomize ./overlay/
- Prints the rendered YAML to stdout only
- Use to inspect the merged manifest
- Redirect to a file for review or diff

Kustomize — Base + Overlays



One **base**, many **overlays**. apply -k and kubectl kustomize are built into kubectl — no extra binary.

DAY 1 · KUSTOMIZE

Kustomize: Base + Overlays, namePrefix, images, patches, kubectl apply -k

LAB 7

Kustomize

Use Kustomize (built into kubectl) to manage base manifests and environment-specific overlays without file duplication. CKA tests kubectl apply -k, namePrefix, image overrides, and strategic merge patches.

You'll build

Create base (deployment + service + kustomization.yaml), dev overlay with namePrefix and image override, production overlay with replica patch. Apply with kubectl apply -k.

Key commands: kubectl kustomize · mkdir -p · cat <<'EOF' · kubectl create · kubectl diff · » namePrefix

LAB 7 · COMMANDS

Kustomize: Base + Overlays, namePrefix, images, patches, kubectl apply -k

Step 1 -- Verify kustomize is built into kubectl

```
kubectl kustomize --help
kubectl version --client -o yaml | grep -i kustomize
```

Step 2 -- Create base layer

```
mkdir -p ~/kustomize-lab/base ~/kustomize-lab/overlays/dev
# Create base/deployment.yaml, base/service.yaml
# Create base/kustomization.yaml with resources list
kubectl kustomize ~/kustomize-lab/base
```

LAB 7 · COMMANDS

Kustomize: Base + Overlays, namePrefix, images, patches, kubectl apply -k (cont.)

Step 3 -- Dev overlay with namePrefix

```
cat <<'EOF' > ~/kustomize-lab/overlays/dev/kustomization.yaml
bases:
  - ../../base
namePrefix: dev-
namespace: dev
images:
  - name: nginx
    newTag: '1.25-alpine'
EOF
kubectl kustomize ~/kustomize-lab/overlays/dev
```

Step 4 -- Apply overlays

```
kubectl create namespace dev
kubectl apply -k ~/kustomize-lab/overlays/dev
kubectl get deployment -n dev
# NAME: dev-webapp
```

LAB 7 · COMMANDS

Kustomize: Base + Overlays, namePrefix, images, patches, kubectl apply -k (cont.)

Step 5 -- Preview changes before applying

```
kubectl diff -k ~/kustomize-lab/overlays/dev
```

Note namePrefix automatically updates all cross-references (selectors, labels). kubectl diff -k previews changes before applying -- essential for safe production.

LAB 7 · TEST IT

Kustomize: Base + Overlays, namePrefix, images, patches, kubectl apply -k

Test it

kubectl get deployment -n dev shows dev-webapp (namePrefix applied). kubectl diff -k shows planned changes without modifying the cluster.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Tea Break

15 minutes

TOPIC 7

RBAC — Role-Based Access Control

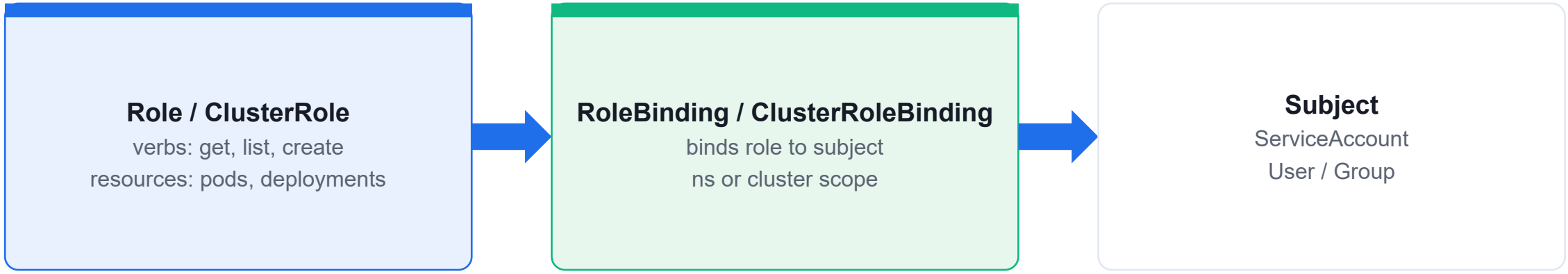
Who can do what, where

AUTHORIZATION

Authorisation in Kubernetes

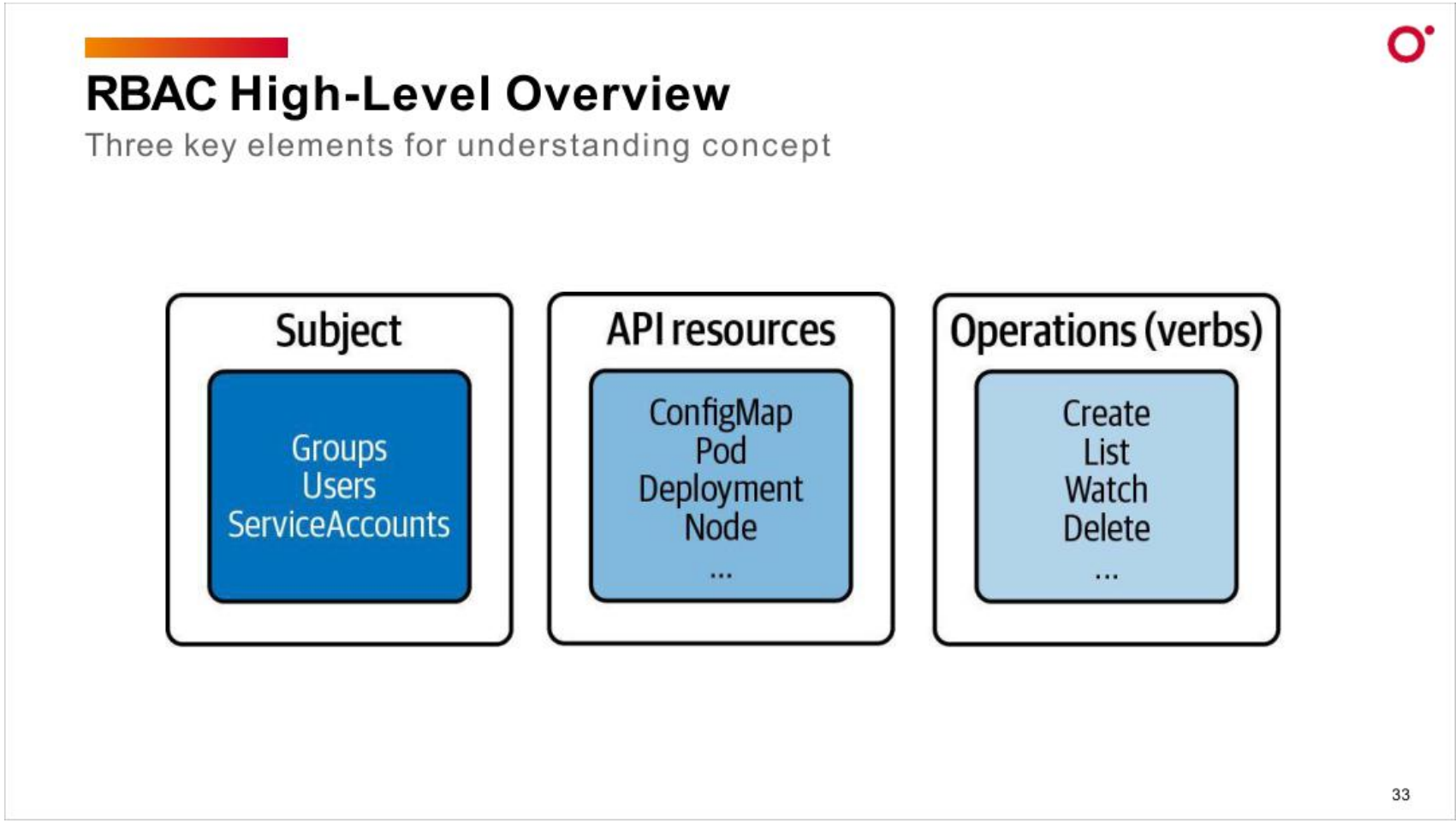
- A Role (namespace) or ClusterRole (cluster-wide) lists allowed verbs on resources.
- A RoleBinding or ClusterRoleBinding ties a Role to a subject: ServiceAccount, user or group.
- ClusterRole + RoleBinding restricts cluster-level role verbs to a single namespace.
- Validate without impersonation: kubectl auth can-i <verb> <resource> --as=<identity>.

RBAC — Role, Binding, Subject




```
kubectl auth can-i list pods --as=system:serviceaccount:ns:sa
→ yes | no
```

RBAC — Subject, API Resources and Verbs



RBAC: Roles, RoleBindings, ServiceAccounts, ClusterRole

LAB 8

RBAC

Create a ServiceAccount for a read-only viewer persona, define a namespaced Role that allows only get/list/watch on Pods, bind the role to the SA, and prove that the SA can read but cannot write. Then extend to ClusterRole for cluster-scoped resources.

You'll build

Create SA + Role + RoleBinding in rbac-demo namespace, test with kubectl auth can-i --as, verify from inside a pod, add ClusterRole for node access.

Key commands: kubectl create · kubectl -n · » Identity

LAB 8 · COMMANDS

RBAC: Roles, RoleBindings, ServiceAccounts, ClusterRole

Step 1 -- Create namespace and ServiceAccount

```
kubectl create ns rbac-demo
kubectl -n rbac-demo create serviceaccount viewer
```

Step 2 -- Create a namespaced Role

```
kubectl -n rbac-demo create role pod-viewer \
  --verb=get,list,watch \
  --resource=pods
kubectl -n rbac-demo get role pod-viewer -o yaml
```

Step 3 -- Bind Role to ServiceAccount

```
kubectl -n rbac-demo create rolebinding viewer-binding \
  --role=pod-viewer \
  --serviceaccount=rbac-demo:viewer
```

LAB 8 · COMMANDS

RBAC: Roles, RoleBindings, ServiceAccounts, ClusterRole (cont.)

Step 4 -- Test with kubectl auth can-i

```
kubectl -n rbac-demo auth can-i list pods \
  --as=system:serviceaccount:rbac-demo:viewer
kubectl -n rbac-demo auth can-i create pods \
  --as=system:serviceaccount:rbac-demo:viewer
# Expected: yes, no
```

Step 5 -- ClusterRole for cluster-scoped resources

```
kubectl create clusterrole node-reader --verb=get,list,watch --resource=nodes
kubectl create clusterrolebinding viewer-nodes \
  --clusterrole=node-reader \
  --serviceaccount=rbac-demo:viewer
```

Note Identity format for SA: system:serviceaccount:<namespace>:<name>. ClusterRole is for cluster-scoped resources (nodes, PVs); Role is namespace-scoped.

LAB 8 · TEST IT

RBAC: Roles, RoleBindings, ServiceAccounts, ClusterRole

Test it

kubectl auth can-i list pods --as=SA returns yes; create pods returns no. ClusterRole allows viewer SA to list nodes across all namespaces.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

WRAP-UP

Day 1 done — you can build and administer a Kubernetes cluster.

Tomorrow: CRDs, workloads, scheduling, services and networking.

COURSE SLIDES

Certified Kubernetes Administrator (CKA)

WSQ Course Code: TGS-2025054612

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

02

DAY 2

Workloads, Scheduling & Services

Domain 1 tools + Domains 2 & 3 start — Labs 9–18

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

Minimum 75% attendance required

DAY 2 OBJECTIVES

Recap & Today

- Yesterday (Domain 1): kubeadm cluster bootstrap, CNI, cluster upgrade, HA etcd, Helm, Kustomize and RBAC.
- This morning — Domain 1 tools: CRDs, Operators and the CRI/CNI/CSI extension interfaces.
- Then Domain 2 (15%): Deployments, ConfigMaps, Secrets, HPA, self-healing and pod scheduling.
- Late afternoon — Domain 3 start (20%): pod connectivity, Service types.

TOPIC 1

CRDs, Operators & Extension Interfaces

Extend Kubernetes without patching it

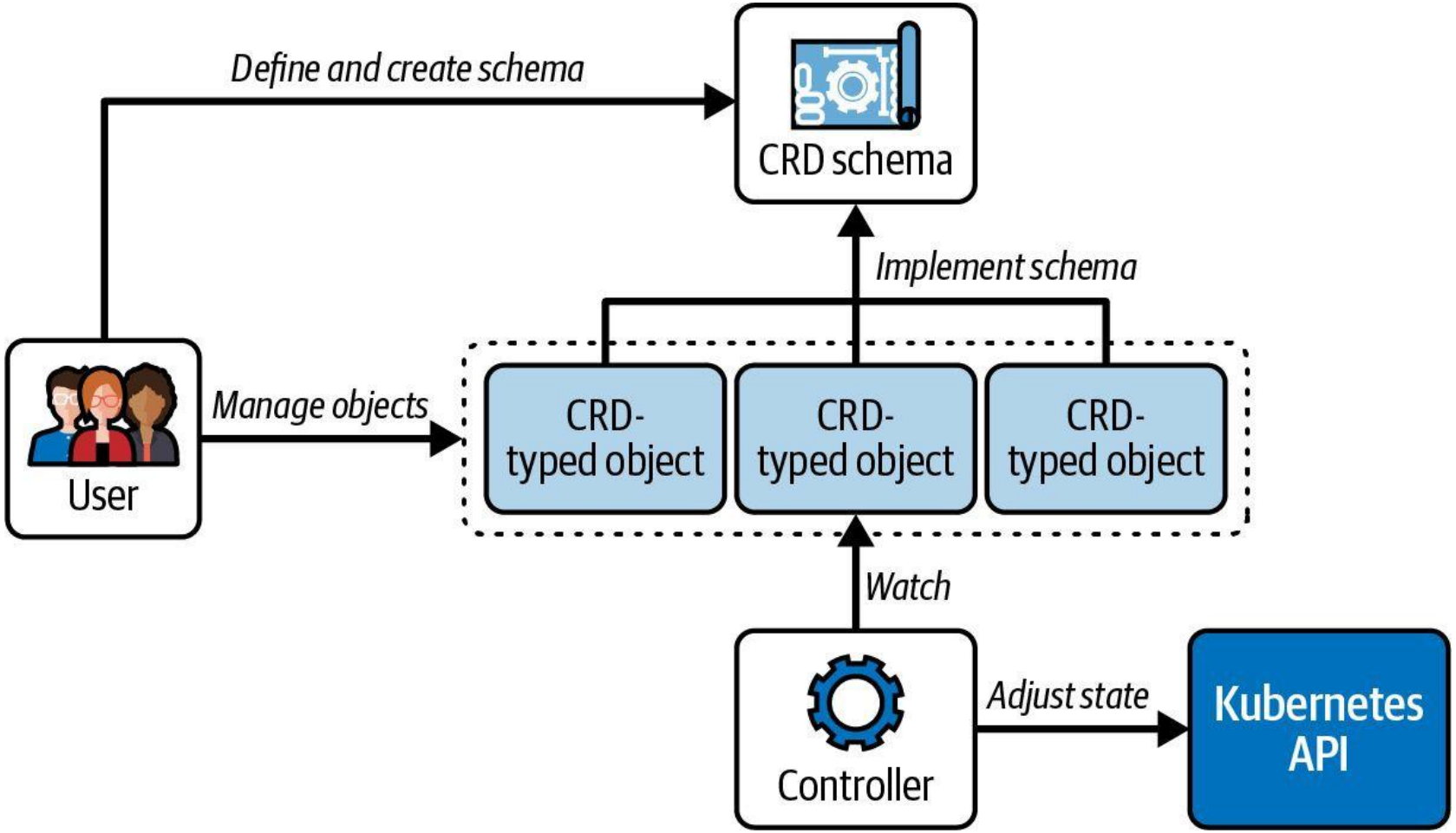
Custom Resource Definitions (CRDs)

- A CRD registers a new object kind with the API server — creating, listing and deleting it just like built-in resources.
- openAPIV3Schema in the CRD spec validates the custom resource at admission time.
- kubectl api-resources lists all resources including custom ones (shows APIVERSION column).
- An Operator pairs a CRD with a controller that encodes operational knowledge (install, upgrade, backup).

CRI, CNI, CSI — The Plugin Interfaces

- CRI (Container Runtime Interface): kubelet talks to containerd or CRI-O via a gRPC socket.
- crictl is the low-level CRI client — use it when kubectl is unavailable because the API server is down.
- CNI (Container Network Interface): /etc/cni/net.d/ configs + /opt/cni/bin/ binaries; called on every pod lifecycle event.
- CSI (Container Storage Interface): kubectl get csidrivers and csinodes; StorageClass references a CSI provisioner.

The Operator Pattern — CRD + Controller



DAY 2 · CRDS & OPERATORS

CRDs and Operators: Schema Validation, cert-manager, Certificate Lifecycle

LAB 9 CRDs and Operators

Define a CRD with OpenAPI schema validation, create custom resources, install the cert-manager operator via Helm, and walk through the TLS certificate lifecycle. CKA tests working with CRDs and operator-managed resources.

You'll build

Create AppDatabase CRD with enum/pattern/required validation, install cert-manager, create self-signed ClusterIssuer, request TLS Certificate, inspect resulting Secret.

Key commands: cat <<'EOF' · kubectl apply · helm repo · » CRDs

LAB 9 · COMMANDS

CRDs and Operators: Schema Validation, cert-manager, Certificate Lifecycle

Step 1 -- Define a CRD with schema validation

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: appdatabases.storage.example.com
spec:
  group: storage.example.com
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            required: ['engine', 'storage']
            properties:
              engine:
                type: string
                enum: ['postgresql', 'mysql', 'redis']
  scope: Namespaced
  names:
    plural: appdatabases
    singular: appdatabase
    kind: AppDatabase
EOF
```

LAB 9 · COMMANDS

CRDs and Operators: Schema Validation, cert-manager, Certificate Lifecycle (cont.)

Step 2 -- Create and query custom resources

```
kubectl apply -f - <<'EOF'
apiVersion: storage.example.com/v1
kind: AppDatabase
metadata:
  name: my-postgres
spec:
  engine: postgresql
  storage: 20Gi
EOF
kubectl get appdatabases
kubectl describe adb my-postgres
```

Step 3 -- Install cert-manager via Helm

```
helm repo add jetstack https://charts.jetstack.io && helm repo update
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --version v1.15.0 --set installCRDs=true
kubectl get pods -n cert-manager
```

LAB 9 · COMMANDS

CRDs and Operators: Schema Validation, cert-manager, Certificate Lifecycle (cont.)

Step 4 -- Create ClusterIssuer and request certificate

```
kubectl apply -f - <<'EOF'
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: selfsigned-issuer
spec:
  selfSigned: {}
EOF
kubectl wait --for=condition=Ready clusterissuer/selfsigned-issuer
```

Note CRDs extend the Kubernetes API with custom resource types. OpenAPI schema validation rejects invalid resources at the API server before they reach etcd.

LAB 9 · TEST IT

CRDs and Operators: Schema Validation, cert-manager, Certificate Lifecycle

Test it

kubectl get appdatabases shows my-postgres. cert-manager pods are Running. ClusterIssuer is Ready. Invalid engine value (e.g. 'oracle') is rejected by the API server.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

DAY 2 · CRI/CNI/CSI

Extension Interfaces: CRI (crictl), CNI (/etc/cni/net.d), CSI (kubectl get csidrivers)

LAB 10 CRI/CNI/CSI extension interfaces

Explore the three Kubernetes extension interfaces using native debugging tools: crictl for CRI, config file inspection for CNI, and kubectl for CSI drivers. Critical for troubleshooting pods stuck in ContainerCreating or volume mount failures.

You'll build

Configure crictl.yaml, list containers/images with crictl, inspect CNI config, list CSI drivers, trace a pod network setup with veth pairs.

Key commands: cat <<'EOF' · POD_ID=$(crictl pods · ls -la · kubectl get · » crictl

LAB 10 · COMMANDS

Extension Interfaces: CRI (crictl), CNI (/etc/cni/net.d), CSI (kubectl get csidrivers)

Step 1 -- Configure and use crictl

```
cat <<'EOF' > /etc/crictl.yaml
runtime-endpoint: unix:///run/containerd/containerd.sock
image-endpoint: unix:///run/containerd/containerd.sock
timeout: 10
EOF
crictl info && crictl pods && crictl ps && crictl images
```

Step 2 -- Get container logs with crictl

```
POD_ID=$(crictl pods --name coredns -q | head -1)
CONTAINER_ID=$(crictl ps --pod $POD_ID -q | head -1)
crictl logs --tail=20 $CONTAINER_ID
```

Step 3 -- Inspect CNI configuration

```
ls -la /etc/cni/net.d/
cat /etc/cni/net.d/10-calico.conflist
ls /opt/cni/bin/
```

LAB 10 · COMMANDS

Extension Interfaces: CRI (crictl), CNI (/etc/cni/net.d), CSI (kubectl get csidrivers) (cont.)

Step 4 -- Inspect CSI drivers

```
kubectl get csidrivers
kubectl get csinodes
kubectl get volumeattachments
```

Note crictl logs works even when kubectl is unavailable (API server down). CNI config: lowest-numbered file in /etc/cni/net.d/ is used. Missing CNI binary in /opt/cni/bin/ causes ContainerCreating to hang indefinitely.

LAB 10 · TEST IT

Extension Interfaces: CRI (crictl), CNI (/etc/cni/net.d), CSI (kubectl get csidrivers)

Test it

crictl pods lists all running pods. CNI config file exists in /etc/cni/net.d/. kubectl get csidrivers returns available storage drivers.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Tea Break

15 minutes

TOPIC 2

Deployments & ReplicaSets

Desired-state workload management

DESIRED STATE

Why Deployments?

- A bare Pod is not rescheduled if it dies — you need a Deployment to maintain the desired count.
- A Deployment manages a ReplicaSet, which manages Pods — the three-tier ownership chain.
- kubectl scale changes replica count; kubectl set image triggers a rolling rollout.
- Old ReplicaSets are retained for rollback (revisionHistoryLimit default 10); rollout undo reverts in one command.

THE OWNERSHIP CHAIN

Deployment → ReplicaSet → Pod

Deployment

- You declare desired state
- replicas + Pod template
- Owns one active ReplicaSet

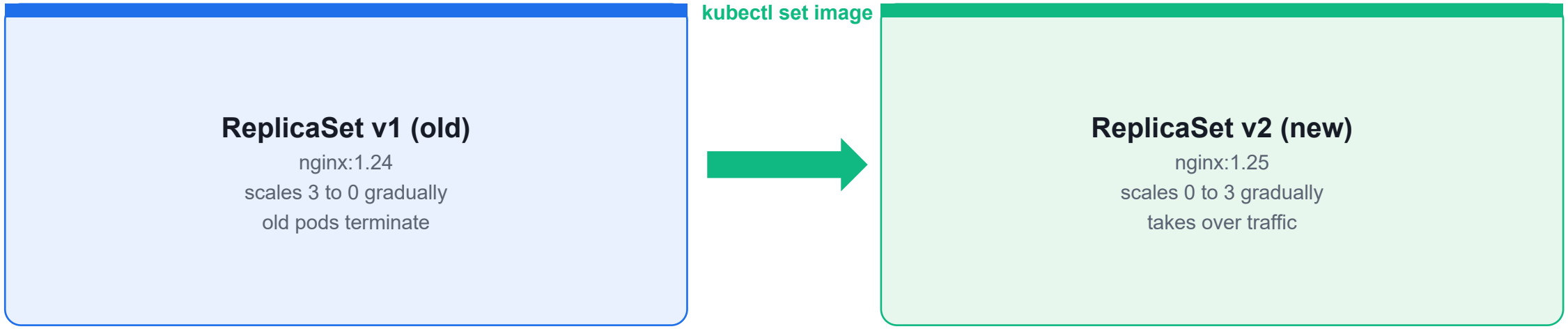
ReplicaSet

- Keeps N Pods running
- Hash-suffixed name
- Self-heals deleted Pods

Pod

- The running workload
- Recreated if it dies
- Never edited directly

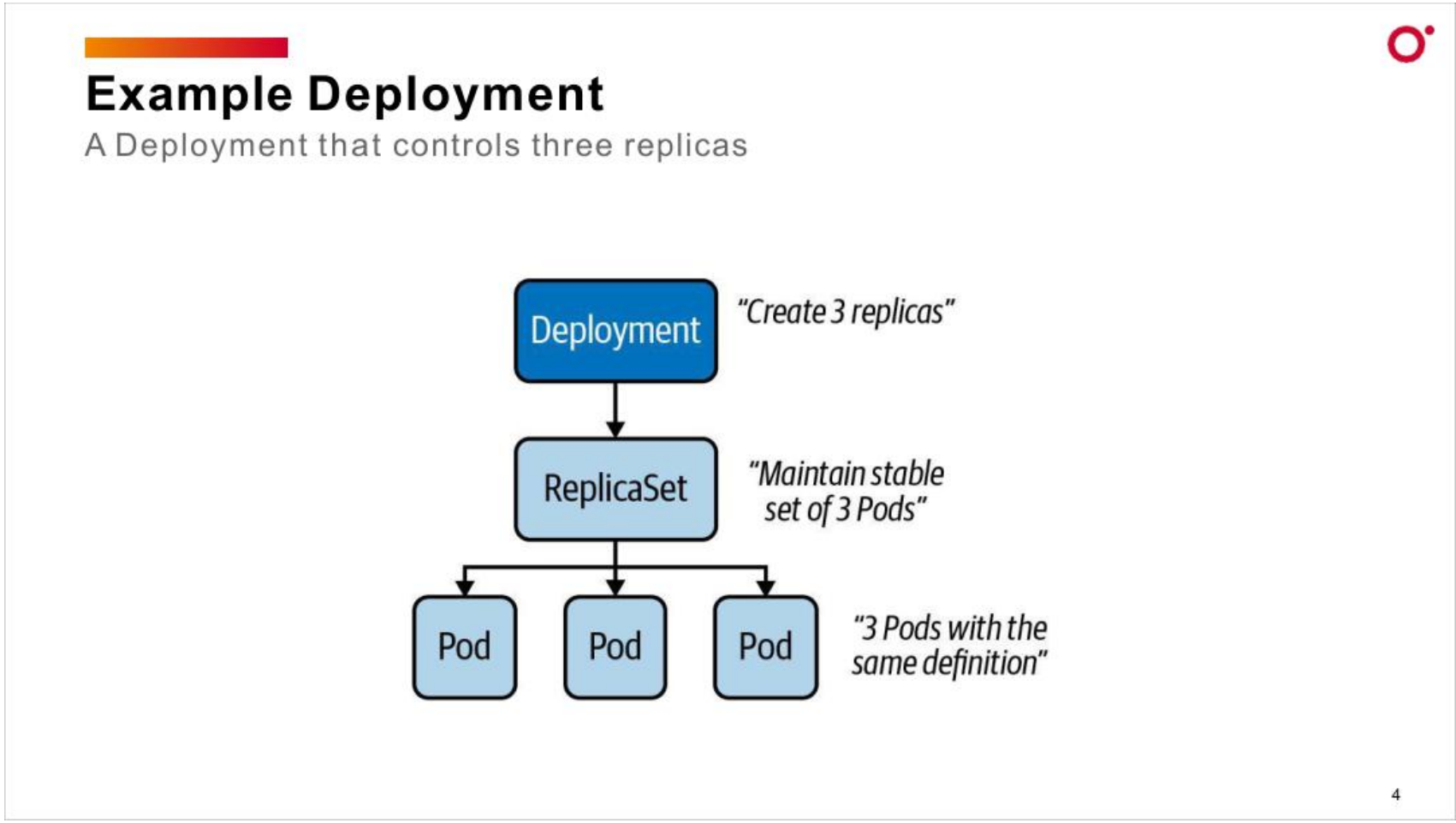
Rolling Update — Old RS to New RS



Old ReplicaSet scales down as the new one scales up — **maxSurge** / **maxUnavailable** control speed. rollout undo reverts in one step.

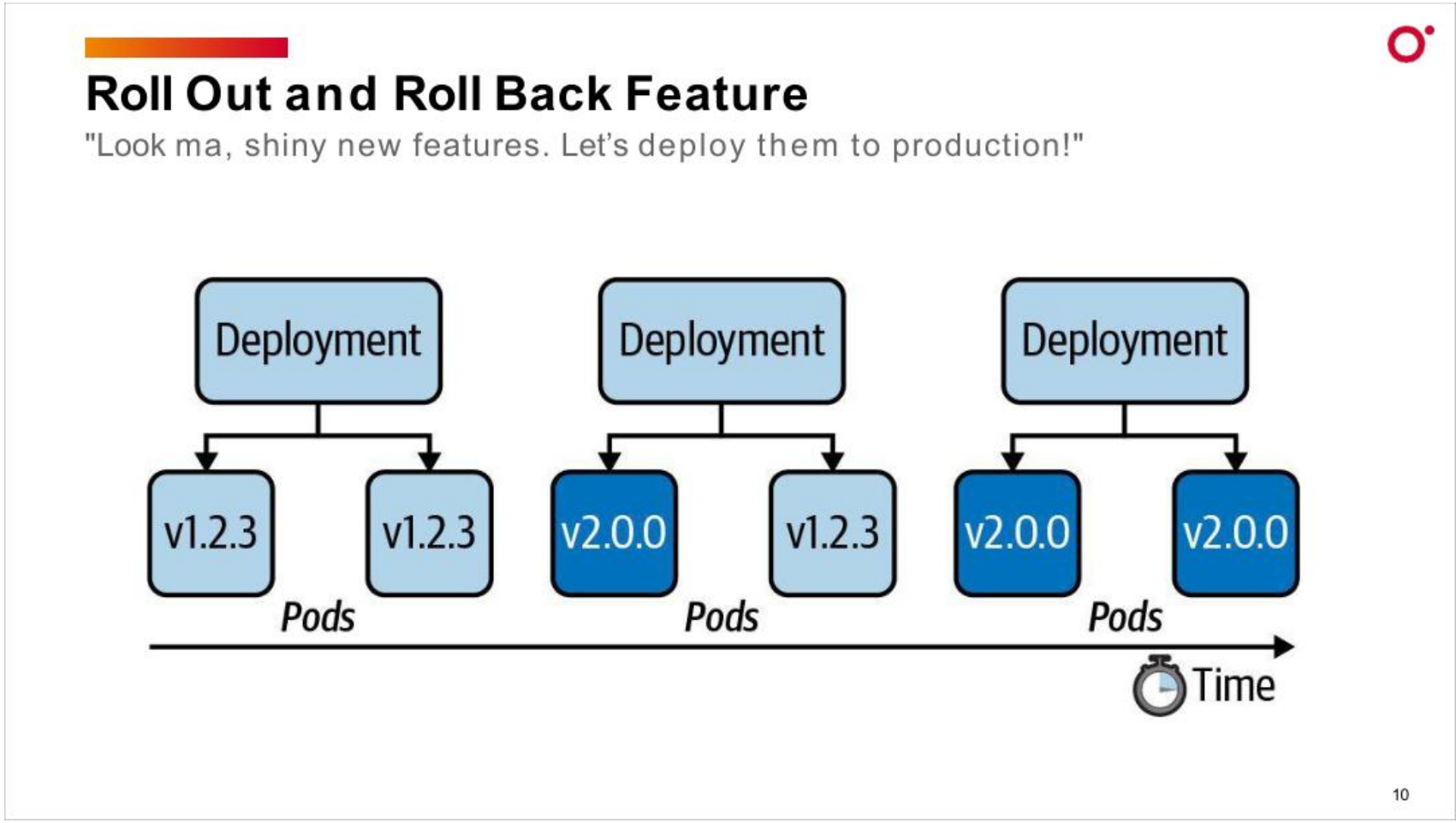
DESIRED STATE

Deployment → ReplicaSet → Pod Hierarchy



DESIRED STATE

Rolling Update and Rollback — Version Transition Over Time



Deployments and Rollouts: Rolling Update, Bad Image, rollout undo, pause/resume

LAB 11 Deployments and rollouts

Create a Deployment, perform a rolling update, deliberately introduce a broken image to trigger a failed rollout, roll back using kubectl rollout undo, and use pause/resume to batch multiple changes. Mastering kubectl rollout subcommands is essential for the 15% Workloads & Scheduling domain.

You'll build

Create 4-replica nginx Deployment, update to new image, break it with invalid tag, rollback, roll back to specific revision, pause/resume.

Key commands: kubectl create · kubectl set · kubectl rollout · » Rolling

LAB 11 · COMMANDS

Deployments and Rollouts: Rolling Update, Bad Image, rollout undo, pause/resume

Step 1 -- Create and expose Deployment

```
kubectl create deployment webapp --image=nginx:1.25 --replicas=4 --port=80
kubectl expose deployment webapp --port=80
kubectl rollout history deployment webapp
```

Step 2 -- Perform a rolling update

```
kubectl set image deployment/webapp webapp=nginx:1.27
kubectl annotate deployment webapp kubernetes.io/change-cause='upgrade to nginx:1.27'
kubectl rollout status deployment webapp
```

Step 3 -- Simulate a broken rollout

```
kubectl set image deployment/webapp webapp=nginx:does-not-exist-999
kubectl get pods -l app=webapp
# Old pods Running; new pod ImagePullBackOff
```

LAB 11 · COMMANDS

Deployments and Rollouts: Rolling Update, Bad Image, rollout undo, pause/resume (cont.)

Step 4 -- Roll back

```
kubectl rollout undo deployment webapp
kubectl rollout status deployment webapp
kubectl rollout undo deployment webapp --to-revision=1
```

Step 5 -- Pause and resume a rollout

```
kubectl set image deployment/webapp webapp=nginx:1.27
kubectl rollout pause deployment webapp
kubectl set resources deployment webapp --containers=webapp --requests=cpu=100m
kubectl rollout resume deployment webapp
kubectl rollout status deployment webapp
```

Note Rolling updates are safe: broken image stalls rollout without taking down healthy pods. kubectl rollout undo creates a NEW revision -- the counter always grows.

LAB 11 · TEST IT

Deployments and Rollouts: Rolling Update, Bad Image, rollout undo, pause/resume

Test it

kubectl rollout history shows multiple revisions. After rollback all pods run previous image. Service continues responding during the bad image period.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Lunch Break

1 hour

TOPIC 3

ConfigMaps & Secrets

Decouple configuration from container images

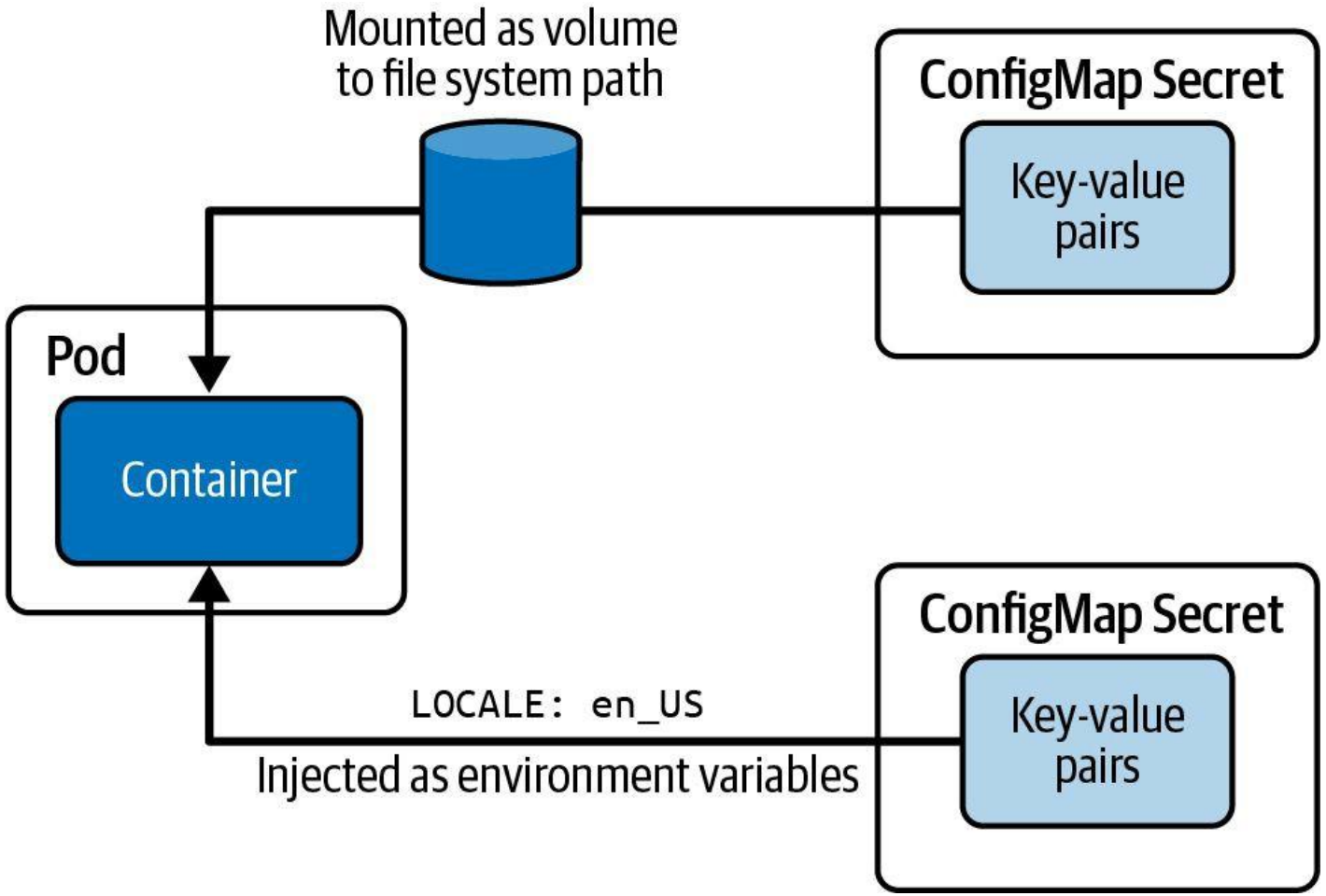
ConfigMaps — Non-secret Config

- A ConfigMap holds non-sensitive key/value data; create with --from-literal, --from-file or --from-env-file.
- Inject one key with configMapKeyRef, all keys with envFrom: configMapRef, or mount as a file volume.
- Volume-mounted ConfigMaps update live (~60 s); env-var injections are fixed at Pod startup.
- Keep configuration out of the image so the same image runs unchanged in every environment.

Secrets — Sensitive Data

- Secrets work like ConfigMaps but values are base64-encoded and access is gated by RBAC.
- Types: generic (Opaque), kubernetes.io/tls and kubernetes.io/dockerconfigjson.
- Inject with envFrom: secretRef (all keys) or secretKeyRef (one key); mount with defaultMode: 0400.
- Base64 is encoding, not encryption — enable encryption at rest and restrict access with RBAC.

Consuming Configuration Data — ConfigMap & Secret



ConfigMaps: 3 Creation Methods, envFrom, Volume Mount, Live Update Semantics

LAB 12 ConfigMaps

Create ConfigMaps using three methods (literal, file, YAML manifest), inject via envFrom and env.valueFrom, mount as volumes with custom filenames and permissions, and demonstrate the live-update behavior difference. Understanding live vs static updates is a tested CKA concept.

You'll build

Create app-config from literals, file-config from files, manifest-config from YAML. Test envFrom with prefix, selective keys, volume mount, subPath, and live update.

Key commands: kubectl create · envFrom: · volumeMounts: · kubectl patch · » CRITICAL:

LAB 12 · COMMANDS

ConfigMaps: 3 Creation Methods, envFrom, Volume Mount, Live Update Semantics

Step 1 -- Create ConfigMap three ways

```
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info
kubectl create configmap file-config --from-file=/tmp/nginx.conf
kubectl get configmap app-config -o yaml
```

Step 2 -- Inject with envFrom

```
# Pod spec:
envFrom:
- configMapRef:
    name: app-config
  prefix: 'CONFIG_'
kubectl exec envfrom-pod -- env | grep CONFIG_
```

LAB 12 · COMMANDS

ConfigMaps: 3 Creation Methods, envFrom, Volume Mount, Live Update Semantics (cont.)

Step 3 -- Mount as volume

```
# Pod spec:
volumeMounts:
- name: config-vol
  mountPath: /config
volumes:
- name: config-vol
  configMap:
    name: app-config
kubectl exec volume-mount-pod -- ls /config
kubectl exec volume-mount-pod -- cat /config/LOG_LEVEL
```

Step 4 -- Live update semantics

```
kubectl patch configmap app-config --type=merge -p '{"data":{"LOG_LEVEL":"debug"}}'
sleep 90
kubectl exec volume-mount-pod -- cat /config/LOG_LEVEL
# debug -- updated live!
kubectl exec envfrom-pod -- env | grep LOG_LEVEL
# info -- NOT updated
```

ConfigMaps: 3 Creation Methods, envFrom, Volume Mount, Live Update Semantics (cont.)

Note CRITICAL: Volume-mounted ConfigMaps update automatically (1-2 min delay). Env vars and subPath mounts require pod restart to pick up changes.

LAB 12 · TEST IT

ConfigMaps: 3 Creation Methods, envFrom, Volume Mount, Live Update Semantics

Test it

volume-mount-pod shows updated LOG_LEVEL=debug after ConfigMap patch. envfrom-pod still shows LOG_LEVEL=info (not live-updated).

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

DAY 2 · SECRETS

Secrets: generic/tls/docker-registry, Env Inject, tmpfs Mount defaultMode 0400

LAB 13

Secrets

Create secrets of three types (generic, TLS, docker-registry), inject via environment variables and volume mounts backed by tmpfs, and apply restrictive permissions with defaultMode 0400. Understand that Secrets are base64-encoded but NOT encrypted by default.

You'll build

Create db-credentials (generic), webapp-tls (TLS with openssl), registry secret. Mount with defaultMode 0400, verify tmpfs, decode with base64 -d.

Key commands: kubectl create · openssl req · volumes: · » Secrets

LAB 13 · COMMANDS

Secrets: generic/tls/docker-registry, Env Inject, tmpfs Mount defaultMode 0400

Step 1 -- Create and decode generic secret

```
kubectl create secret generic db-credentials \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=SuperSecret123!
kubectl get secret db-credentials -o yaml
kubectl get secret db-credentials \
  -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
```

Step 2 -- Create TLS secret

```
openssl req -x509 -nodes -newkey rsa:2048 \
  -keyout /tmp/tls.key -out /tmp/tls.crt -days 365 \
  -subj '/CN=webapp.example.com'
kubectl create secret tls webapp-tls --cert=/tmp/tls.crt --key=/tmp/tls.key
```

LAB 13 · COMMANDS

Secrets: generic/tls/docker-registry, Env Inject, tmpfs Mount defaultMode 0400 (cont.)

Step 3 -- Mount as tmpfs with defaultMode 0400

```
# Pod spec volumes:
volumes:
- name: secret-vol
  secret:
    secretName: db-credentials
    defaultMode: 0400
kubectl exec secret-volume-pod -- ls -la /secrets
# -r-------- files
kubectl exec secret-volume-pod -- df -h /secrets
# tmpfs
```

Note Secrets are base64-encoded (NOT encrypted). Enable etcd encryption at rest for actual security. tmpfs means secrets are never written to node disk. echo -n is critical -- without -n, trailing newline corrupts the base64 value.

LAB 13 · TEST IT

Secrets: generic/tls/docker-registry, Env Inject, tmpfs Mount defaultMode 0400

Test it

Secret files have permissions -r-------- (0400). df -h /secrets shows tmpfs. base64 -d decodes the password correctly. TLS secret type is kubernetes.io/tls.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Tea Break

15 minutes

TOPIC 4

HPA & Self-Healing Workloads

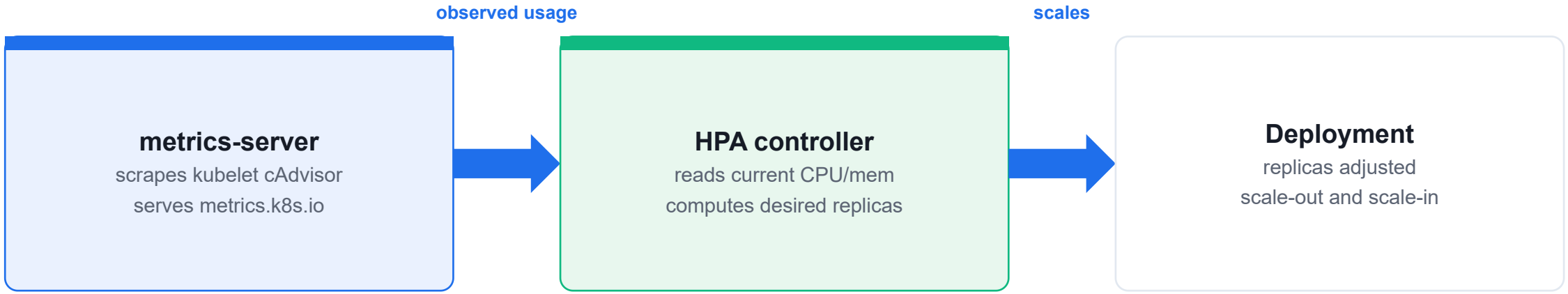
Autoscaling and resilient workload patterns

AUTOSCALING

Horizontal Pod Autoscaler

- HPA watches metrics-server and adjusts the Deployment replica count to hit a target CPU/memory %.
- Pods must have resources.requests defined for HPA to calculate a utilisation ratio.
- Scale-out is fast; scale-in has a stabilisation window (~5 min) to prevent thrashing.
- Create with: kubectl autoscale deployment <name> --min=2 --max=10 --cpu-percent=50.

Horizontal Pod Autoscaler



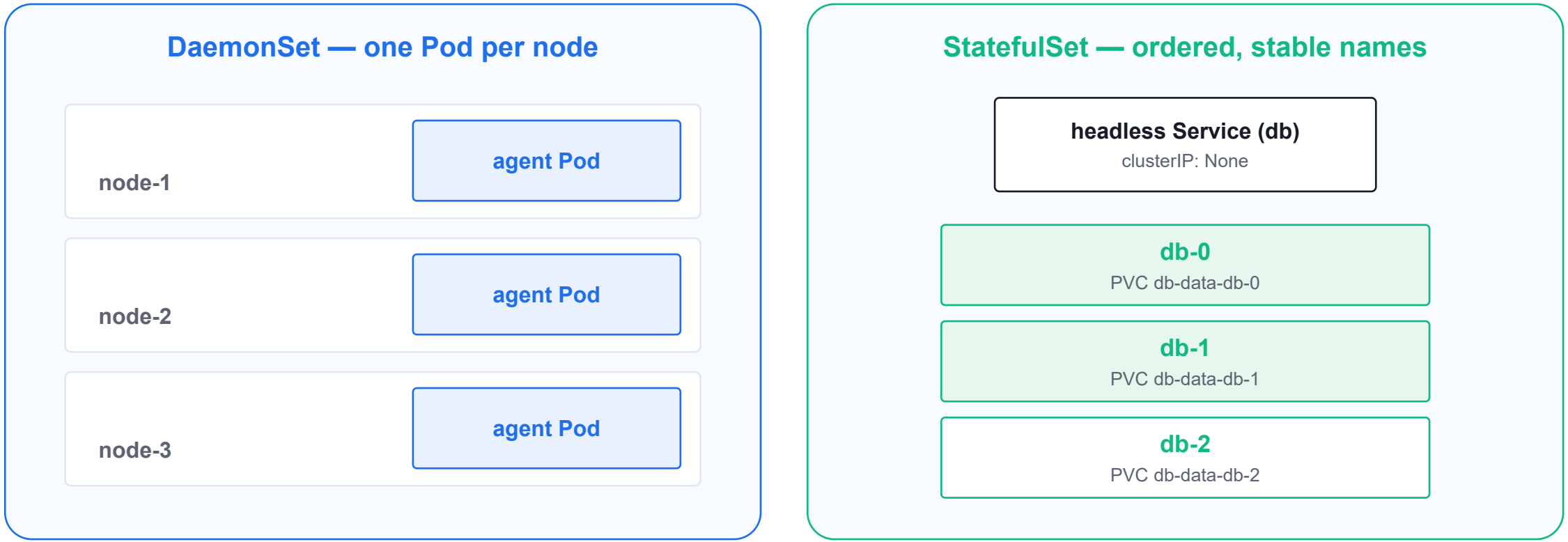
HPA requires **resources.requests** on the Deployment. Scale-in waits ~5 min (stabilization window).

RESILIENCE

Self-Healing Primitives (DaemonSet, StatefulSet)

- Liveness probe: restarts the container when it hangs or deadlocks (httpGet / tcpSocket / exec).
- Readiness probe: removes the pod from Service endpoints until it can serve traffic — no restart.
- DaemonSet: schedules exactly one pod per matching node — for log collectors, CNI agents, node exporters.
- StatefulSet: stable names (db-0, db-1), ordered start/stop, volumeClaimTemplates — for databases and queues.

DaemonSet vs StatefulSet



DaemonSet scales with the cluster; StatefulSet gives **stable network identity** and ordered start/stop — essential for databases.

DAY 2 · HPA

HPA v2: metrics-server, HorizontalPodAutoscaler, Load Test, Watch Scale

LAB 14 Horizontal Pod Autoscaling

Install metrics-server to provide CPU metrics, create an HPA using autoscaling/v2 API, generate CPU load, and watch the HPA scale up then back down. HPA requires resource requests on containers; without them, utilization cannot be calculated.

You'll build

Install metrics-server (with --kubelet-insecure-tls patch), create HPA targeting 50% CPU, run stress test, watch scale-up, wait for scale-down.

Key commands: kubectl apply · kubectl create · kubectl autoscale · kubectl run · » HPA

LAB 14 · COMMANDS

HPA v2: metrics-server, HorizontalPodAutoscaler, Load Test, Watch Scale

Step 1 -- Install metrics-server

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl patch deployment metrics-server -n kube-system --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
until kubectl top node 2>/dev/null; do echo 'waiting...'; sleep 5; done
```

Step 2 -- Deployment with resource requests

```
kubectl create deployment cpu-app --image=nginx --replicas=1
kubectl set resources deployment cpu-app --containers=nginx \
  --requests=cpu=100m --limits=cpu=500m
```

Step 3 -- Create HPA

```
kubectl autoscale deployment cpu-app --min=1 --max=10 --cpu-percent=50
kubectl get hpa && kubectl describe hpa cpu-app
```

LAB 14 · COMMANDS

HPA v2: metrics-server, HorizontalPodAutoscaler, Load Test, Watch Scale (cont.)

Step 4 -- Generate load and watch scaling

```
kubectl run load-gen --image=busybox --restart=Never \
  -- sh -c 'while true; do wget -q -O- http://cpu-app; done'
kubectl get hpa cpu-app -w
kubectl get pods -w
```

Note HPA requires resource requests -- without them metrics-server cannot calculate % utilization. Scale-down has a 5-minute cooldown by default.

LAB 14 · TEST IT

HPA v2: metrics-server, HorizontalPodAutoscaler, Load Test, Watch Scale

Test it

kubectl get hpa shows non-zero CPU% and increasing replicas under load. After removing load-gen, replicas scale back to 1 within 5-10 minutes.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

DAY 2 · SELF-HEALING

Self-Healing: Liveness/Readiness Probes, ReplicaSet, DaemonSet, StatefulSet

LAB 15

Self-healing workloads

Configure liveness and readiness probes to control container health, demonstrate ReplicaSet self-healing when pods are manually deleted, deploy a DaemonSet that runs one pod per node, and create a StatefulSet with stable network identity.

You'll build

Deploy pod with httpGet liveness probe, show ReplicaSet maintains desired count, deploy nginx DaemonSet, create StatefulSet with volumeClaimTemplates.

Key commands: cat <<'EOF' · readinessProbe: · kubectl create · » Liveness:

LAB 15 · COMMANDS

Self-Healing: Liveness/Readiness Probes, ReplicaSet, DaemonSet, StatefulSet

Step 1 -- Liveness probe: httpGet

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: liveness-http
spec:
  containers:
  - name: webapp
    image: nginx:alpine
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 10
      failureThreshold: 3
EOF
kubectl get pod liveness-http -w
```

LAB 15 · COMMANDS

Self-Healing: Liveness/Readiness Probes, ReplicaSet, DaemonSet, StatefulSet (cont.)

Step 2 -- Readiness probe blocks traffic

```
# Readiness probe -- pod not added to Service endpoints until probe passes
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 10
```

Step 3 -- ReplicaSet self-healing

```
kubectl create deployment selfheal --image=nginx --replicas=3
kubectl delete pod -l app=selfheal --force
kubectl get pods -l app=selfheal -w
# New pods created immediately
```

LAB 15 · COMMANDS

Self-Healing: Liveness/Readiness Probes, ReplicaSet, DaemonSet, StatefulSet (cont.)

Step 4 -- DaemonSet: one pod per node

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-monitor
spec:
  selector:
    matchLabels:
      app: node-monitor
  template:
    metadata:
      labels:
        app: node-monitor
    spec:
      containers:
      - name: monitor
        image: busybox
        command: ['sleep', '3600']
EOF
kubectl get daemonset
kubectl get pods -l app=node-monitor -o wide
```

LAB 15 · COMMANDS

Self-Healing: Liveness/Readiness Probes, ReplicaSet, DaemonSet, StatefulSet (cont.)

Note Liveness: fail -> container restart. Readiness: fail -> removed from Service endpoints. DaemonSet automatically places one pod per node, including newly joined nodes.

LAB 15 · TEST IT

Self-Healing: Liveness/Readiness Probes, ReplicaSet, DaemonSet, StatefulSet

Test it

Pod restarts when liveness probe fails. kubectl delete pod does not reduce ReplicaSet count. DaemonSet has exactly one pod per node.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

TOPIC 5

Pod Scheduling

Taints, tolerations, node affinity and resource requests

WHERE PODS LAND

Scheduling Controls

- nodeSelector is the simplest control — schedule only on nodes with matching key=value labels.
- Taints repel pods without a matching toleration; effects are NoSchedule, PreferNoSchedule and NoExecute.
- nodeAffinity allows required (hard) and preferred (soft) constraints using In/NotIn/Exists operators.
- resources.requests tell the scheduler how much CPU/memory a pod needs; limits cap consumption at runtime.

Pod Scheduling Controls

nodeSelector
spec.nodeSelector: {gpu: 'true'}

→ Schedules only on nodes with matching labels — simplest scheduling control.

Taints & Tolerations
kubectl taint node <n> key=val:NoSchedule

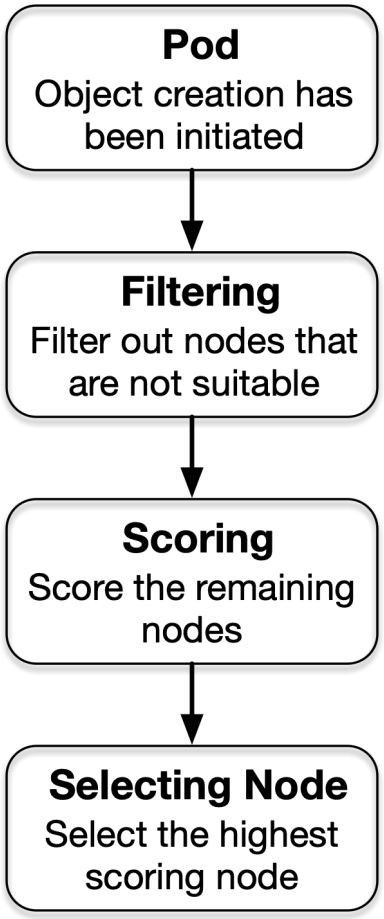
→ Node repels pods without a matching toleration. NoExecute also evicts running pods.

nodeAffinity
requiredDuringSchedulingIgnoredDuringExecution

→ Hard (required) or soft (preferred) rules. Operators: In / NotIn / Exists / Gt / Lt.

Also: podAffinity / podAntiAffinity · resources.requests drive scheduling · priorityClass pre-empts lower-priority pods

Pod Scheduling Algorithm — Filter, Score, Select Node



DAY 2 · SCHEDULING

Scheduling: nodeSelector, Node Affinity, Taints + Tolerations, Resource Requests

LAB 16

Scheduling

Control pod placement using nodeSelector for simple label matching, node affinity for flexible required/preferred placement rules, taints to repel pods from nodes, and tolerations to allow specific pods through taints.

You'll build

Label nodes, create pod with nodeSelector, add required/preferred node affinity, taint a node (NoSchedule), add toleration, demonstrate resource-based scheduling.

Key commands: kubectl label · kubectl taint · » Taint

LAB 16 · COMMANDS

Scheduling: nodeSelector, Node Affinity, Taints + Tolerations, Resource Requests

Step 1 -- nodeSelector: simple label matching

```
kubectl label node worker01 disk=ssd
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: ssd-pod
spec:
  nodeSelector:
    disk: ssd
  containers:
  - name: app
    image: nginx
EOF
kubectl get pod ssd-pod -o wide
```

LAB 16 · COMMANDS

Scheduling: nodeSelector, Node Affinity, Taints + Tolerations, Resource Requests (cont.)

Step 2 -- Node Affinity

```
# requiredDuringSchedulingIgnoredDuringExecution: hard constraint
# preferredDuringSchedulingIgnoredDuringExecution: soft preference
# Check: kubectl get pod affinity-pod -o wide
```

Step 3 -- Taints and Tolerations

```
kubectl taint node worker01 special=gpu:NoSchedule
kubectl run blocked --image=nginx
kubectl get pod blocked -o wide
# Stays Pending -- no toleration
# Add toleration:
tolerations:
- key: special
  operator: Equal
  value: gpu
  effect: NoSchedule
```

LAB 16 · COMMANDS

Scheduling: nodeSelector, Node Affinity, Taints + Tolerations, Resource Requests (cont.)

Step 4 -- Remove taint and clean up

```
kubectl taint node worker01 special=gpu:NoSchedule-
kubectl get pods
```

Note Taint effects: NoSchedule (hard reject), PreferNoSchedule (soft reject), NoExecute (evict existing pods). Tolerations do NOT guarantee placement on the tainted node.

LAB 16 · TEST IT

Scheduling: nodeSelector, Node Affinity, Taints + Tolerations, Resource Requests

Test it

nodeSelector pod lands on worker01 (disk=ssd). Tainted node rejects pods without toleration. Pod with matching toleration schedules on the tainted node.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

TOPIC 6

Pod Connectivity & Services

How Pods communicate and are exposed

Pod-to-Pod Connectivity

- Every pod gets a unique IP from the pod CIDR; all pods can reach each other without NAT (flat network).
- The CNI plugin implements this — without it, cross-node pod traffic fails.
- Pod IPs are ephemeral — use a Service for stable addressing across restarts and scaling.
- kubectl exec <pod> -- curl <podIP>:<port> verifies direct pod-level connectivity bypassing the Service.

Service Types

ClusterIP
default — virtual IP only inside cluster

→ stable DNS + VIP for pod-to-pod; not reachable from outside

NodePort
exposes port 30000-32767 on every node

→ external access without a cloud LB; good for bare-metal

LoadBalancer
provisions cloud/MetalLB external IP

→ production external access; builds on NodePort

kubectl get endpoints <svc> — verify the Service is selecting the right pods.

Pod Connectivity: Flat Pod Network, ping, curl, DNS Discovery, netshoot

LAB 17 Pod networking

Verify pod-to-pod communication using ping and curl, test DNS-based service discovery, trace network paths with traceroute, and use nicolaka/netshoot as a network debugging toolkit. Pod networking fundamentals are the foundation for the 20% Services & Networking domain.

You'll build

Deploy server-pod and client-pod, ping by Pod IP, curl by Service DNS name, nslookup kubernetes.default, traceroute between pods.

Key commands: kubectl run · SERVER_IP=$(kubectl get · kubectl exec · » DNS

LAB 17 · COMMANDS

Pod Connectivity: Flat Pod Network, ping, curl, DNS Discovery, netshoot

Step 1 -- Deploy test pods

```
kubectl run server-pod --image=nginx --labels='role=server'
kubectl expose pod server-pod --port=80 --name=server-svc
kubectl run client-pod --image=nicolaka/netshoot --command -- sleep 3600
kubectl get pods -o wide
```

Step 2 -- Ping by Pod IP

```
SERVER_IP=$(kubectl get pod server-pod -o jsonpath='{.status.podIP}')
kubectl exec client-pod -- ping -c 4 $SERVER_IP
```

Step 3 -- DNS-based service discovery

```
kubectl exec client-pod -- curl -s http://server-svc.default.svc.cluster.local
kubectl exec client-pod -- nslookup kubernetes.default.svc.cluster.local
kubectl exec client-pod -- cat /etc/resolv.conf
```

LAB 17 · COMMANDS

Pod Connectivity: Flat Pod Network, ping, curl, DNS Discovery, netshoot (cont.)

Step 4 -- Network debugging with netshoot

```
kubectl exec client-pod -- traceroute $SERVER_IP
kubectl exec client-pod -- dig server-svc.default.svc.cluster.local
kubectl exec client-pod -- ss -tulpn
```

Note DNS pattern: <service>.<namespace>.svc.cluster.local. nicolaka/netshoot includes ping, curl, dig, traceroute, nmap, tshark, tcpdump.

LAB 17 · TEST IT

Pod Connectivity: Flat Pod Network, ping, curl, DNS Discovery, netshoot

Test it

Ping to server-pod IP returns 0% loss. curl to service DNS returns nginx HTML. nslookup kubernetes.default resolves to the ClusterIP of the kubernetes service.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Service Types: ClusterIP, NodePort, LoadBalancer (MetalLB), EndpointSlices, Selector Debug

LAB 18 Service types

Create and test all three primary service types -- ClusterIP, NodePort, and LoadBalancer with MetalLB -- inspect EndpointSlice objects, and debug selector mismatches that cause services to have no endpoints. Service types are heavily tested (20% CKA domain).

You'll build

Create ClusterIP, NodePort with custom port, inspect EndpointSlices, create service with wrong selector and debug/fix it.

Key commands: kubectl create · kubectl expose · » EndpointSlices

LAB 18 · COMMANDS

Service Types: ClusterIP, NodePort, LoadBalancer (MetalLB), EndpointSlices, Selector Debug

Step 1 -- ClusterIP service

```
kubectl create deployment webapp --image=nginx --replicas=3
kubectl expose deployment webapp --port=80 --type=ClusterIP
kubectl get svc webapp
kubectl get endpointslices -l kubernetes.io/service-name=webapp
```

Step 2 -- NodePort service

```
kubectl expose deployment webapp --port=80 --type=NodePort --name=webapp-np
NODE_PORT=$(kubectl get svc webapp-np -o jsonpath='{.spec.ports[0].nodePort}')
echo "NodePort: $NODE_PORT"
```

LAB 18 · COMMANDS

Service Types: ClusterIP, NodePort, LoadBalancer (MetalLB), EndpointSlices, Selector Debug (cont.)

Step 3 -- Debug: selector mismatch causing no endpoints

```
# Create service with wrong selector
kubectl expose deployment webapp --port=80 --name=broken-svc
kubectl patch svc broken-svc -p '{"spec":{"selector":{"app":"wrong"}}}'
kubectl get endpoints broken-svc
# <none> -- no matching pods
# Fix: correct the selector
kubectl patch svc broken-svc -p '{"spec":{"selector":{"app":"webapp"}}}'
```

Note EndpointSlices replace Endpoints in Kubernetes v1.21+. A service with no endpoints usually means a selector label mismatch. Compare: kubectl describe svc vs kubectl get pods --show-labels.

LAB 18 · TEST IT

Service Types: ClusterIP, NodePort, LoadBalancer (MetalLB), EndpointSlices, Selector Debug

Test it

ClusterIP routes traffic to all 3 pods. NodePort is accessible from node IP. broken-svc has no endpoints; fixing selector restores them immediately.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

WRAP-UP

Day 2 done — workloads, scheduling and basic services.

Tomorrow: Ingress, Gateway API, NetworkPolicy, Storage and Troubleshooting.

COURSE SLIDES

Certified Kubernetes Administrator (CKA)

WSQ Course Code: TGS-2025054612

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

03

DAY 3

Networking, Storage & Troubleshooting

Domains 3, 4 & 5 — Labs 19–28

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

Minimum 75% attendance required

DAY 3 OBJECTIVES

Recap & Today

- Yesterday: CRDs, extension interfaces, Deployments, ConfigMaps, Secrets, HPA, scheduling and Services.
- Morning — Domain 3 remaining (20%): Ingress, Gateway API, NetworkPolicy, CoreDNS.
- Afternoon — Domain 4 (10%): PV/PVC, StorageClass, dynamic provisioning.
- Late afternoon — Domain 5 start (30%): cluster component failures, node troubleshooting, pod debugging.

TOPIC 1

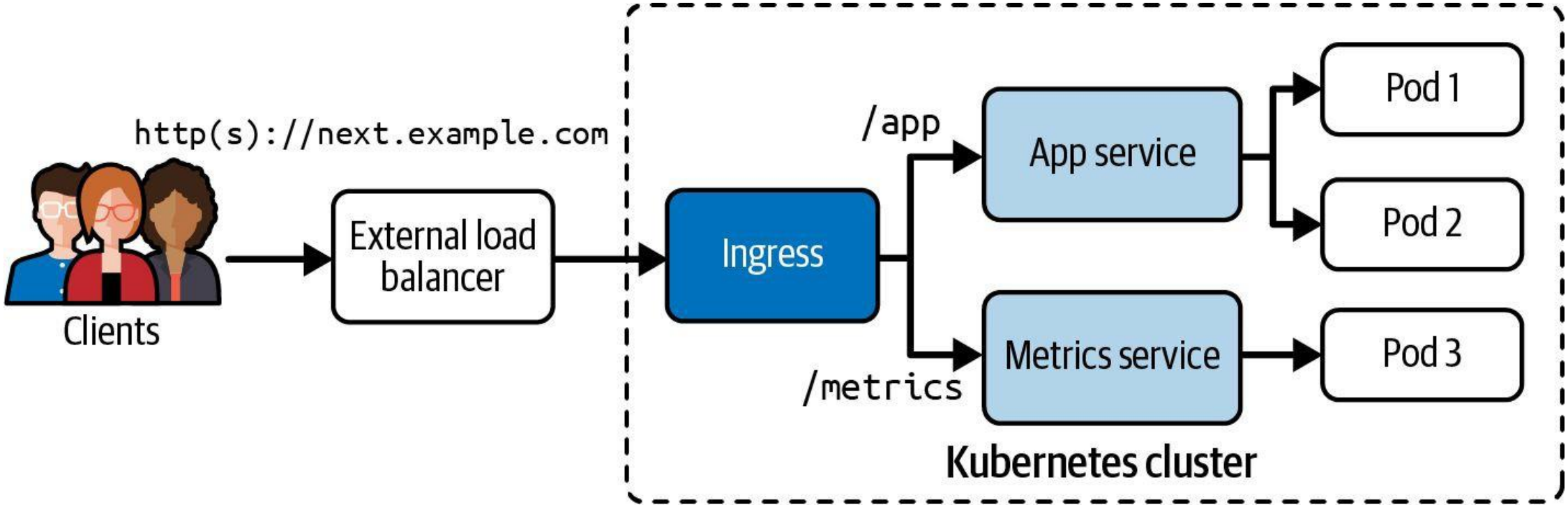
Ingress & Gateway API

Layer-7 HTTP/HTTPS routing into the cluster

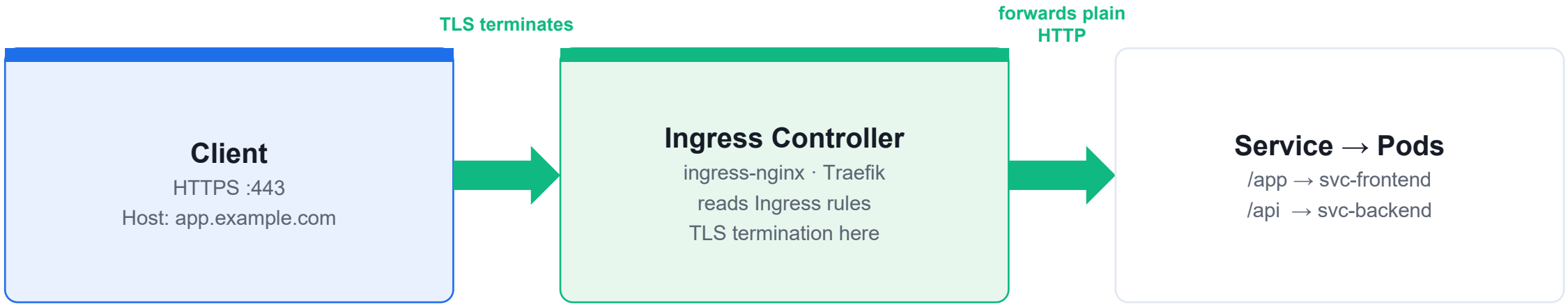
Ingress — L7 Routing

- An Ingress defines host- and path-based HTTP/HTTPS rules that forward traffic to backend Services.
- An Ingress controller (e.g. ingress-nginx) must be running; ingressClassName selects it.
- TLS terminates at the controller — tls.secretName points at a kubernetes.io/tls Secret.
- pathType: Prefix matches a path and everything below it; Exact requires a literal match.

Ingress Traffic Routing — Host/Path to Service



Ingress — L7 HTTP/HTTPS Routing



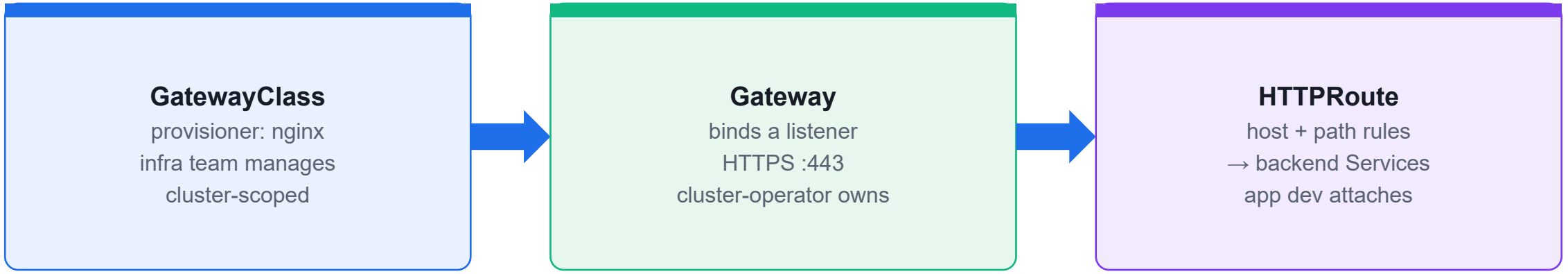
ingressClassName: nginx · tls.secretName: tls-secret · pathType: Prefix | Exact

apiVersion: networking.k8s.io/v1 (extensions/v1beta1 removed in 1.22). **A controller must be running** — ingressClassName selects it.

Gateway API — The Modern Alternative

- Gateway API is the successor to Ingress — role-oriented (infra / cluster-operator / app dev).
- GatewayClass defines the controller; Gateway binds a listener; HTTPRoute attaches rules.
- Supports HTTP, HTTPS, TCP, TLS and gRPC routing with cross-namespace route attachment.
- apiVersion: gateway.networking.k8s.io/v1 — install the CRDs separately before using it.

Gateway API — Role-Oriented Routing



apiVersion: gateway.networking.k8s.io/v1 · **install CRDs first** → kubectl apply -f standard-install.yaml from the Gateway API releases.

DAY 3 · INGRESS

Ingress Controller: Host Routing, TLS Termination, Path Rules, Debug

LAB 19

Ingress

Install ingress-nginx controller, configure host-based routing to multiple backends, terminate TLS using a Kubernetes Secret, and debug misconfigured Ingress resources. CKA 2026 tests installing ingress-nginx, host routing, and TLS termination.

You'll build

Install ingress-nginx via bare-metal manifest, create Ingress for two apps with host rules, add TLS block with secret, debug 404 by checking ingressClassName.

Key commands: kubectl apply · HTTP_PORT=$(kubectl -n · cat <<'EOF' · kubectl create · » ingressClassName:

LAB 19 · COMMANDS

Ingress Controller: Host Routing, TLS Termination, Path Rules, Debug

Step 1 -- Install ingress-nginx

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/baremetal/deploy.yaml
kubectl -n ingress-nginx wait --for=condition=Ready pod \
  -l app.kubernetes.io/component=controller --timeout=180s
```

Step 2 -- Get NodePort for testing

```
HTTP_PORT=$(kubectl -n ingress-nginx get svc ingress-nginx-controller \
  -o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}')
echo "HTTP NodePort: $HTTP_PORT"
```

Ingress Controller: Host Routing, TLS Termination, Path Rules, Debug (cont.)

Step 3 -- Create Ingress with host-based routing

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multi-host
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: app1.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app1-svc
            port:
              number: 80
EOF
```

LAB 19 · COMMANDS

Ingress Controller: Host Routing, TLS Termination, Path Rules, Debug (cont.)

Step 4 -- TLS termination

```
kubectl create secret tls tls-secret --cert=tls.crt --key=tls.key
# Add to Ingress spec:
# tls:
# - hosts: [app1.example.com]
#   secretName: tls-secret
```

Note ingressClassName: nginx is required in Kubernetes 1.18+. Without it, the Ingress is ignored. Check ingress-nginx logs: kubectl logs -n ingress-nginx -l app.kubernetes.io/component=controller

LAB 19 · TEST IT

Ingress Controller: Host Routing, TLS Termination, Path Rules, Debug

Test it

curl -H 'Host: app1.example.com' http://NODE:PORT returns app1 content. Wrong ingressClassName causes 404. TLS ingress terminates HTTPS correctly.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Gateway API: GatewayClass, Gateway, HTTPRoute (GA in v1.28, tested in CKA 2026)

LAB 20 Gateway API

Use the Kubernetes Gateway API (GA as of v1.28, tested in CKA 2026) to route traffic using GatewayClass, Gateway, and HTTPRoute objects. Gateway API is role-oriented: infrastructure providers manage GatewayClass/Gateway; developers manage HTTPRoutes.

You'll build

Install Gateway API CRDs and NGINX Gateway Fabric, create GatewayClass and Gateway, define HTTPRoutes with path-based routing.

Key commands: kubectl apply · cat <<'EOF' · » Three

LAB 20 · COMMANDS

Gateway API: GatewayClass, Gateway, HTTPRoute (GA in v1.28, tested in CKA 2026)

Step 1 -- Install Gateway API CRDs

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.1.0/standard-install.yaml
kubectl get crds | grep gateway
```

Step 2 -- Install NGINX Gateway Fabric

```
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-fabric/v1.4.0/deploy/crds.yaml
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-fabric/v1.4.0/deploy/manifests/nginx-gateway.yaml
```

LAB 20 · COMMANDS

Gateway API: GatewayClass, Gateway, HTTPRoute (GA in v1.28, tested in CKA 2026) (cont.)

Step 3 -- Create GatewayClass and Gateway

```
cat <<'EOF' | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx
spec:
  controllerName: gateway.nginx.org/nginx-gateway-controller
---
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: main-gateway
spec:
  gatewayClassName: nginx
  listeners:
  - name: http
    port: 80
    protocol: HTTP
EOF
```

LAB 20 · COMMANDS

Gateway API: GatewayClass, Gateway, HTTPRoute (GA in v1.28, tested in CKA 2026) (cont.)

Step 4 -- Create HTTPRoute

```
cat <<'EOF' | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-route
spec:
  parentRefs:
  - name: main-gateway
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /api
    backendRefs:
    - name: api-svc
      port: 80
EOF
```

LAB 20 · COMMANDS

Gateway API: GatewayClass, Gateway, HTTPRoute (GA in v1.28, tested in CKA 2026) (cont.)

Note Three core Gateway API objects: GatewayClass (infra tier), Gateway (cluster operator tier), HTTPRoute (developer tier). Role separation enables multi-tenant traffic management.

LAB 20 · TEST IT

Gateway API: GatewayClass, Gateway, HTTPRoute (GA in v1.28, tested in CKA 2026)

Test it

kubectl get gatewayclass shows nginx as Accepted. kubectl get gateway shows Programmed=True. HTTP request to /api path routes to api-svc backend.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Tea Break

15 minutes

TOPIC 2

NetworkPolicy & CoreDNS

Network isolation and in-cluster DNS

NetworkPolicy — Allow-List Firewalling

- By default every pod can talk to every other pod — NetworkPolicies apply an allow-list.
- Default-deny: podSelector: {} with policyTypes: [Ingress] and no ingress rules blocks all inbound.
- Allow by podSelector (pod label match) or namespaceSelector (whole namespaces) in the from/to block.
- Policies are additive (OR): multiple policies that match the same pod combine their allow rules.

NetworkPolicy — Default Deny + Allow

Default Deny (all pods)

podSelector: {} + policyTypes: [Ingress]

→ **nothing reaches any pod**

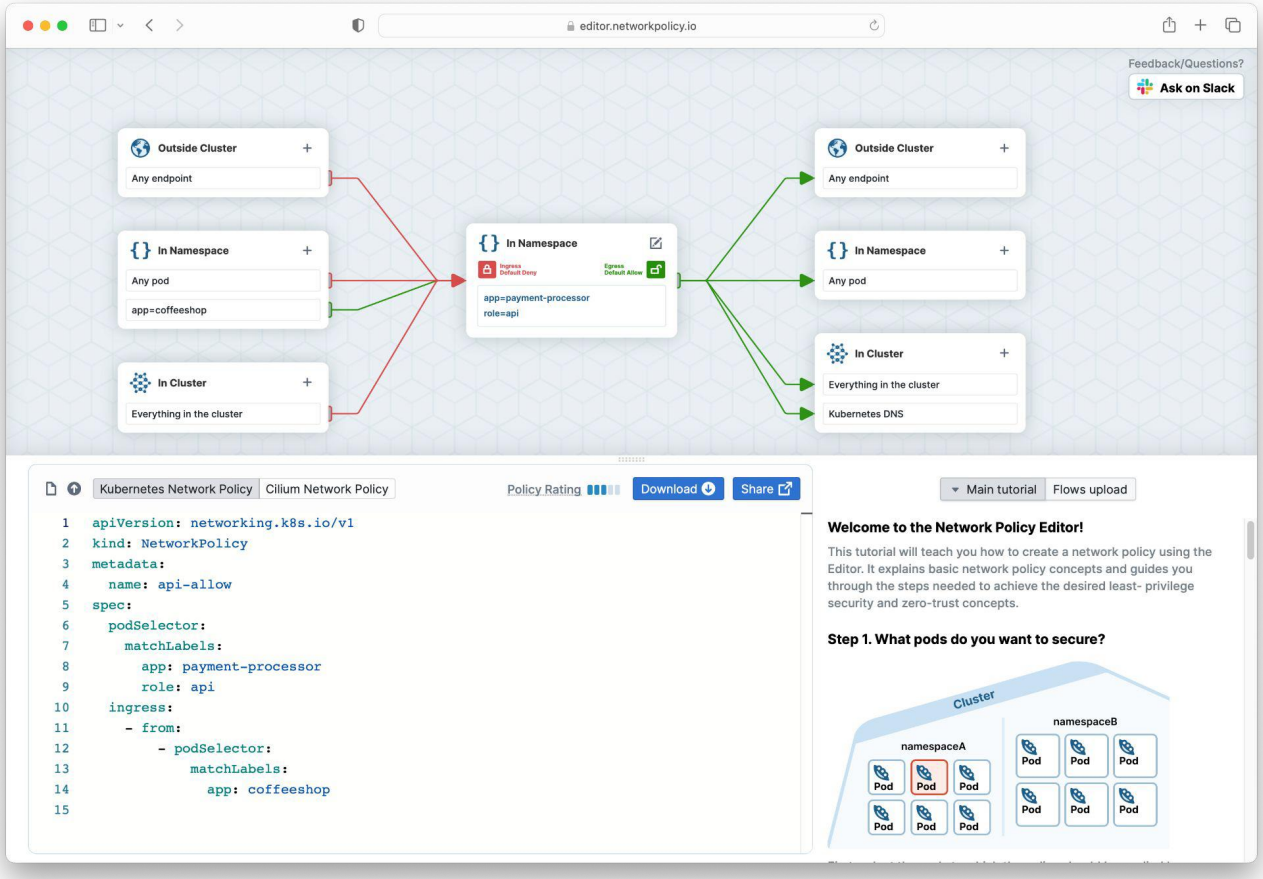
Selective Allow (podSelector)

```
ingress:
  from:
    - podSelector: {role: allowed}
  ports: [{port: 80}]
```

Policies are additive (OR). Combined default-deny + allow = allow-list model.

NETWORK ISOLATION

Visualising Network Policy Rules

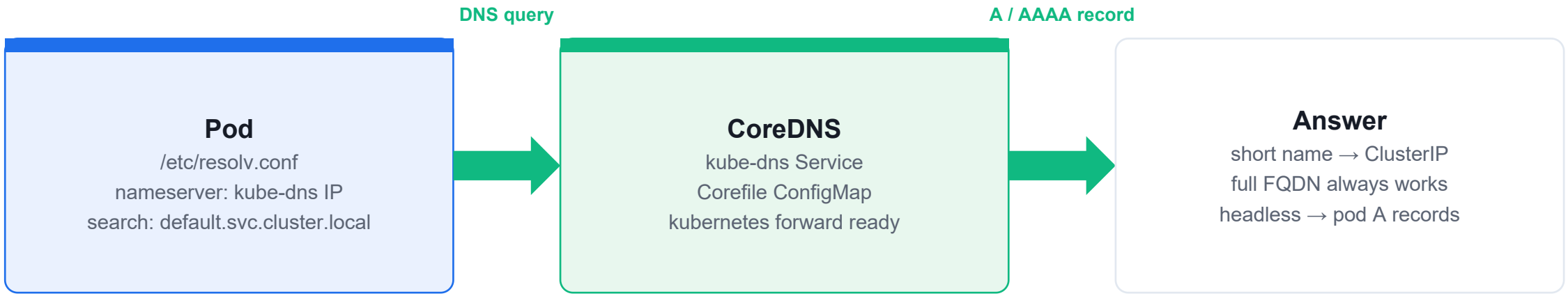


SERVICE DISCOVERY

CoreDNS — In-Cluster DNS

- Every pod's /etc/resolv.conf points to the kube-dns ClusterIP with a search list.
- FQDN format: <service>.<namespace>.svc.cluster.local — within the same namespace the short name resolves.
- Headless Service (clusterIP: None): DNS returns individual pod A records instead of a single virtual IP.
- Corefile ConfigMap controls CoreDNS behaviour — add stub zones, upstream forwarders or rewrite rules.

CoreDNS — In-Cluster DNS Resolution



Within ns: svc · cross-ns: svc.ns · full: svc.ns.svc.cluster.local

Headless Service (clusterIP: None) returns individual **Pod A records** instead of a single virtual IP.

DAY 3 · NETWORK POLICY

NetworkPolicy: Default-Deny, Pod Selector, Namespace Selector, Egress Lockdown

LAB 21

Network Policies

Implement Pod-level firewall rules using NetworkPolicies. Kubernetes uses allow-lists: once any NetworkPolicy selects a pod, all traffic not explicitly allowed is blocked. CKA 2026 tests default-deny, pod/namespace selectors, and egress lockdown.

You'll build

Create server and client pods, apply default-deny-ingress policy, verify blocking, add allow rule for specific pod selector, test egress lockdown.

Key commands: kubectl create · cat <<'EOF' · » Empty

LAB 21 · COMMANDS

NetworkPolicy: Default-Deny, Pod Selector, Namespace Selector, Egress Lockdown

Step 1 -- Create test workloads

```
kubectl create ns netpol
kubectl -n netpol run server --image=nginx --labels='app=server'
kubectl -n netpol run client-ok --image=nicolaka/netshoot \
  --labels='role=allowed' --command -- sleep 3600
kubectl -n netpol run client-bad --image=nicolaka/netshoot \
  --labels='role=denied' --command -- sleep 3600
kubectl -n netpol expose pod server --port=80
```

LAB 21 · COMMANDS

NetworkPolicy: Default-Deny, Pod Selector, Namespace Selector, Egress Lockdown (cont.)

Step 2 -- Apply default-deny-ingress

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: netpol
spec:
  podSelector: {}
  policyTypes: ['Ingress']
EOF
kubectl -n netpol exec client-ok -- curl -m 3 http://server || echo BLOCKED
```

LAB 21 · COMMANDS

NetworkPolicy: Default-Deny, Pod Selector, Namespace Selector, Egress Lockdown (cont.)

Step 3 -- Allow specific pod selector

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-allowed
  namespace: netpol
spec:
  podSelector:
    matchLabels:
      app: server
  policyTypes: ['Ingress']
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: allowed
EOF
kubectl -n netpol exec client-ok -- curl -s http://server
# OK
kubectl -n netpol exec client-bad -- curl -m 3 http://server || echo BLOCKED
```

LAB 21 · COMMANDS

NetworkPolicy: Default-Deny, Pod Selector, Namespace Selector, Egress Lockdown (cont.)

Note Empty podSelector {} selects ALL pods in the namespace. NetworkPolicy requires a CNI that enforces it (Calico, Cilium -- NOT Flannel alone).

LAB 21 · TEST IT

NetworkPolicy: Default-Deny, Pod Selector, Namespace Selector, Egress Lockdown

Test it

default-deny blocks both clients. After allow policy, client-ok reaches server; client-bad (role=denied) is still blocked.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

DAY 3 · COREDNS

CoreDNS: Corefile Inspection, Service DNS, Headless Services, Stub Zones

LAB 22

CoreDNS

Inspect CoreDNS configuration, query Service and Pod DNS records, understand headless service resolution (A records per pod), and customize CoreDNS with stub zones. CKA tests configuring CoreDNS via ConfigMap and diagnosing DNS failures.

You'll build

Inspect kube-dns service and coredns ConfigMap, test DNS from netshoot pod, create headless service, verify per-pod A records, add stub zone to Corefile.

Key commands: kubectl -n · kubectl run · kubectl expose · » Service

LAB 22 · COMMANDS

CoreDNS: Corefile Inspection, Service DNS, Headless Services, Stub Zones

Step 1 -- Inspect CoreDNS

```
kubectl -n kube-system get svc kube-dns
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl -n kube-system get configmap coredns -o yaml
```

Step 2 -- Query DNS from debug pod

```
kubectl run dnsdebug --image=nicolaka/netshoot --command -- sleep 3600
kubectl exec dnsdebug -- cat /etc/resolv.conf
kubectl exec dnsdebug -- nslookup kubernetes.default.svc.cluster.local
kubectl exec dnsdebug -- dig +short webapp.default.svc.cluster.local
```

Step 3 -- Headless service: per-pod A records

```
kubectl expose deployment webapp --port=80 --name=webapp-headless --cluster-ip=None
kubectl exec dnsdebug -- dig +short webapp-headless.default.svc.cluster.local
# Returns one IP per pod -- used by StatefulSets for stable identity
```

LAB 22 · COMMANDS

CoreDNS: Corefile Inspection, Service DNS, Headless Services, Stub Zones (cont.)

Step 4 -- Add stub zone to Corefile

```
kubectl -n kube-system edit configmap coredns
# Add inside Corefile:
# example.internal:53 {
#   forward . 10.10.0.1
# }
```

Note Service name is 'kube-dns' (legacy) even though pods run CoreDNS. Headless services have clusterIP: None. Stub zones forward specific domains to external DNS.

LAB 22 · TEST IT

CoreDNS: Corefile Inspection, Service DNS, Headless Services, Stub Zones

Test it

nslookup kubernetes.default resolves to 10.96.0.1. Headless service dig returns one IP per pod. CoreDNS ConfigMap has the Corefile visible.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Lunch Break

1 hour

TOPIC 3

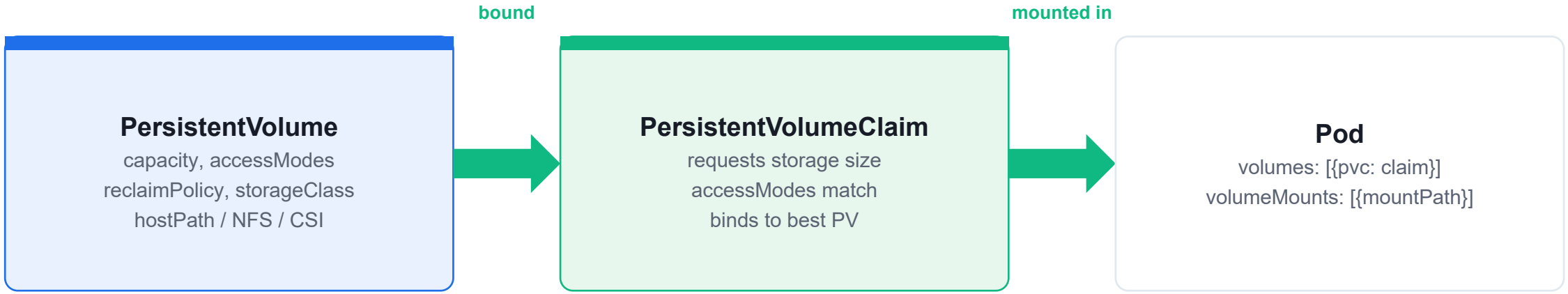
Persistent Storage

PV, PVC, StorageClass and dynamic provisioning

PersistentVolumes and Claims

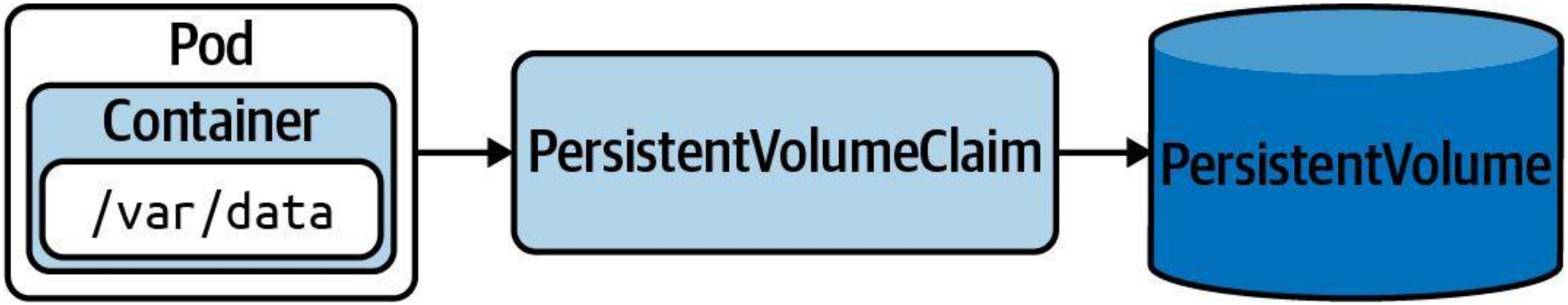
- A PersistentVolume (PV) is a cluster-level storage resource with capacity, accessModes and reclaimPolicy.
- A PersistentVolumeClaim (PVC) is a namespaced request that binds to a matching PV.
- Access modes: ReadWriteOnce (node), ReadOnlyMany (many nodes), ReadWriteMany (many nodes).
- reclaimPolicy Retain keeps data after PVC deletion; Delete removes the underlying volume.

PV, PVC, and Pod



Dynamic: PVC → **StorageClass** → CSI provisioner creates PV on demand. **Retain** keeps data; **Delete** removes it.

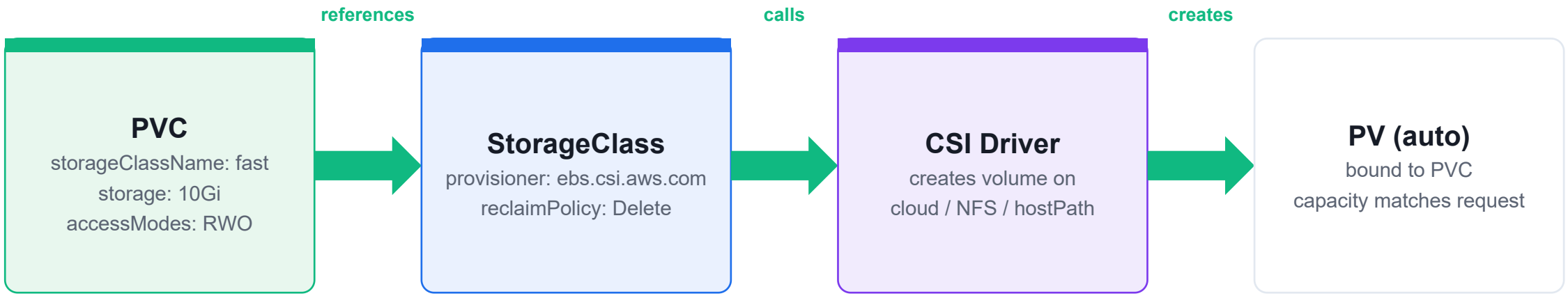
Persistent Volume — Lifecycle Architecture



StorageClass & Dynamic Provisioning

- A StorageClass references a CSI provisioner that creates PVs on demand when a PVC is submitted.
- The default StorageClass is annotated storageclass.kubernetes.io/is-default-class: 'true'.
- PVC without storageClassName uses the default class automatically.
- StatefulSet volumeClaimTemplates give each replica its own PVC — they are not auto-deleted on scale-down.

StorageClass — Dynamic Provisioning



PVC without storageClassName uses the **default StorageClass**. reclaimPolicy Delete removes PV on PVC deletion; Retain keeps data.

PersistentVolume and PVC: Static Provisioning, Access Modes, Reclaim Policies

LAB 23

PersistentVolumes and PersistentVolumeClaims

Create a PersistentVolume (static provisioning), bind it via a PVC, mount in a pod, and understand PV/PVC binding rules including access modes and reclaim policies. CKA tests static provisioning, access modes, and the fact that PVCs outlive pods.

You'll build

Create hostPath PV with ReadWriteOnce, create PVC that binds, mount in nginx pod, write data, delete pod and recreate to verify persistence.

Key commands: sudo mkdir · cat > · cat <<'EOF' · » Access

LAB 23 · COMMANDS

PersistentVolume and PVC: Static Provisioning, Access Modes, Reclaim Policies

Step 1 -- Create a PersistentVolume

```
sudo mkdir -p /mnt/data && echo 'hello from host' | sudo tee /mnt/data/index.html
cat > pv.yaml <<'EOF'
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-host
spec:
  capacity:
    storage: 1Gi
  accessModes: [ReadWriteOnce]
  hostPath:
    path: /mnt/data
  persistentVolumeReclaimPolicy: Retain
EOF
kubectl apply -f pv.yaml && kubectl get pv
```

LAB 23 · COMMANDS

PersistentVolume and PVC: Static Provisioning, Access Modes, Reclaim Policies (cont.)

Step 2 -- Create a PVC that binds to the PV

```
cat > pvc.yaml <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-host
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
EOF
kubectl apply -f pvc.yaml && kubectl get pvc
# STATUS: Bound
```

LAB 23 · COMMANDS

PersistentVolume and PVC: Static Provisioning, Access Modes, Reclaim Policies (cont.)

Step 3 -- Mount PVC in a pod

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: pvc-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: data
      mountPath: /usr/share/nginx/html
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: pvc-host
EOF
kubectl exec pvc-pod -- cat /usr/share/nginx/html/index.html
```

LAB 23 · COMMANDS

PersistentVolume and PVC: Static Provisioning, Access Modes, Reclaim Policies (cont.)

Note Access modes: ReadWriteOnce (one node RW), ReadOnlyMany (many nodes R), ReadWriteMany (many nodes RW). Reclaim: Retain (manual cleanup), Delete (removes storage), Recycle (deprecated).

LAB 23 · TEST IT

PersistentVolume and PVC: Static Provisioning, Access Modes, Reclaim Policies

Test it

PVC shows STATUS=Bound. Pod reads 'hello from host' from the mounted volume. After pod deletion and recreation, data persists (Retain policy).

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

DAY 3 · STORAGECLASS

StorageClass: Dynamic Provisioning, Default Class, StatefulSet volumeClaimTemplates

LAB 24 StorageClass and dynamic provisioning

Use StorageClasses for dynamic PV creation, mark a class as default, and create a StatefulSet with volumeClaimTemplates that provisions one PVC per pod. CKA tests creating StorageClasses and dynamic provisioning.

You'll build

Install local-path-provisioner, mark StorageClass as default, create dynamically-provisioned PVC, create StatefulSet with volumeClaimTemplates.

Key commands: kubectl apply · kubectl patch · cat <<'EOF' · » StatefulSet

LAB 24 · COMMANDS

StorageClass: Dynamic Provisioning, Default Class, StatefulSet volumeClaimTemplates

Step 1 -- Install local-path-provisioner

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-storage.yaml
kubectl -n local-path-storage rollout status deploy/local-path-provisioner
kubectl get storageclass
```

Step 2 -- Mark StorageClass as default

```
kubectl patch storageclass local-path \
  -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

LAB 24 · COMMANDS

StorageClass: Dynamic Provisioning, Default Class, StatefulSet volumeClaimTemplates (cont.)

Step 3 -- Dynamic PVC provisioning

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dynamic-pvc
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: local-path
  resources:
    requests:
      storage: 512Mi
EOF
kubectl get pvc dynamic-pvc && kubectl get pv
```

LAB 24 · COMMANDS

StorageClass: Dynamic Provisioning, Default Class, StatefulSet volumeClaimTemplates (cont.)

Step 4 -- StatefulSet with volumeClaimTemplates

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
spec:
  serviceName: db
  replicas: 3
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
      - name: postgres
        image: postgres:16-alpine
        env:
        - name: POSTGRES_PASSWORD
          value: example
        volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ReadWriteOnce]
```

LAB 24 · COMMANDS

StorageClass: Dynamic Provisioning, Default Class, StatefulSet volumeClaimTemplates (cont.)

Note StatefulSet volumeClaimTemplates create one PVC per pod: data-db-0, data-db-1, data-db-2. PVCs are NOT deleted when the StatefulSet is deleted -- manual cleanup required.

LAB 24 · TEST IT

StorageClass: Dynamic Provisioning, Default Class, StatefulSet volumeClaimTemplates

Test it

dynamic-pvc is Bound. StatefulSet db has 3 pods each with their own PVC (data-db-0, etc).

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Volume Types: emptyDir, hostPath, projected, downwardAPI

LAB 25 Pod volume types

Explore pod-level volume types: emptyDir for shared scratch space, hostPath for node filesystem access, projected for combining multiple sources into one mount, and downwardAPI for exposing pod metadata to the application.

You'll build

Create pod with emptyDir shared between two containers, projected volume combining ConfigMap + Secret, downwardAPI volume exposing pod name/namespace/resource requests.

Key commands: cat <<'EOF' · kubectl exec · » emptyDir

LAB 25 · COMMANDS

Volume Types: emptyDir, hostPath, projected, downwardAPI

Step 1 -- emptyDir: shared scratch space

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: scratch
spec:
  containers:
  - name: writer
    image: busybox
    command: ['sh', '-c', 'echo hello-shared > /data/file; sleep 3600']
    volumeMounts:
    - name: tmp
      mountPath: /data
  - name: reader
    image: busybox
    command: ['sh', '-c', 'sleep 3600']
    volumeMounts:
    - name: tmp
      mountPath: /data
  volumes:
  - name: tmp
    emptyDir: {}
EOF
kubectl exec scratch -c reader -- cat /data/file
```

Volume Types: emptyDir, hostPath, projected, downwardAPI (cont.)

Step 2 -- projected volume (ConfigMap + Secret)

```
# volumes:
# - name: combined
#   projected:
#     sources:
#     - configMap:
#         name: app-config
#     - secret:
#         name: app-secret
```

LAB 25 · COMMANDS

Volume Types: emptyDir, hostPath, projected, downwardAPI (cont.)

Step 3 -- downwardAPI: pod metadata as files

```
# volumes:
# - name: podinfo
#   downwardAPI:
#     items:
#     - path: 'pod-name'
#       fieldRef:
#         fieldPath: metadata.name
#     - path: 'cpu-request'
#       resourceFieldRef:
#         resource: requests.cpu
kubectl exec downward-pod -- cat /podinfo/pod-name
```

Note emptyDir is deleted when the pod is deleted. hostPath ties the pod to a specific node. projected combines ConfigMap, Secret, serviceAccountToken, and downwardAPI into one mount.

LAB 25 · TEST IT

Volume Types: emptyDir, hostPath, projected, downwardAPI

Test it

reader container reads file written by writer via emptyDir. downwardAPI pod shows correct pod name from /podinfo/pod-name.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Tea Break

15 minutes

TOPIC 4

Troubleshooting Cluster Components

etcd, API server, scheduler and controller-manager

Cluster Component Failures

- Static pod manifests live in /etc/kubernetes/manifests/ — kubelet watches this directory and auto-restarts them.
- API server down? kubectl will fail — switch to crictl ps -a / crictl logs and journalctl -u kubelet.
- etcdctl endpoint health verifies etcd before assuming the API server itself is broken.
- Fix a broken manifest, save it, and the kubelet restarts the static pod automatically within ~30 seconds.

DOMAIN 5 · 7-STEP DEBUG CHAIN

Cluster Troubleshooting Triage

1. kubectl get/describe	Read Events — usually tells you the cause directly
2. kubectl logs (--previous)	Read container output; --previous for a crashed pod
3. systemctl / journalctl	On the node: status kubelet containerd; journalctl -u kubelet
4. crictl ps / logs	Use when kubectl is unavailable (API server down)
5. etcdctl endpoint health	Verify etcd is healthy before assuming API server bugs
6. kubectl get events -A	Cluster-wide event stream — sorted by lastTimestamp
7. kubectl auth can-i	Confirm RBAC allows the operation before blaming the app

NODE HEALTH

Node Troubleshooting

- Five node conditions: Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable.
- kubelet stopped → node goes NotReady after ~40 s; systemctl start kubelet restores it.
- Disk pressure: df -h on the node — kubelet evicts pods when thresholds are hit; free space to recover.
- cordon = SchedulingDisabled (no new pods); drain = evict existing pods; uncordon restores scheduling.

Troubleshoot Cluster Components: API Server, Controller Manager, Scheduler, etcd

LAB 26

Control plane troubleshooting

Diagnose and fix broken control plane components. When kubectl is unresponsive, use crictl and journalctl instead. Troubleshooting is the highest-weighted CKA domain at 30%.

You'll build

Locate static pod manifests, introduce a break in kube-apiserver.yaml (wrong flag), diagnose with crictl and journalctl, fix manifest and verify recovery.

Key commands: ls /etc/kubernetes/manifests/ · crictl pods · sudo vi · kubectl get · » Static

LAB 26 · COMMANDS

Troubleshoot Cluster Components: API Server, Controller Manager, Scheduler, etcd

Step 1 -- Know where control plane components live

```
ls /etc/kubernetes/manifests/
sudo systemctl status kubelet --no-pager | head -5
sudo systemctl status containerd --no-pager | head -5
```

Step 2 -- Diagnose API server failure

```
# If kubectl is unresponsive:
crictl pods
crictl ps -a
crictl logs $(crictl ps -a --name kube-apiserver -q)
journalctl -u kubelet --no-pager -n 50 | grep -i error
sudo cat /etc/kubernetes/manifests/kube-apiserver.yaml | head -50
```

LAB 26 · COMMANDS

Troubleshoot Cluster Components: API Server, Controller Manager, Scheduler, etcd (cont.)

Step 3 -- Fix static pod manifest

```
# Wrong flag/image in manifest causes API server to not start
sudo vi /etc/kubernetes/manifests/kube-apiserver.yaml
# Kubelet detects change within 20 seconds
crictl ps | grep apiserver
kubectl get nodes
# Should work once apiserver recovers
```

Step 4 -- Diagnose controller manager and scheduler

```
kubectl get pods -n kube-system | grep -E 'controller|scheduler'
kubectl logs -n kube-system kube-controller-manager-controlplane | tail -20
kubectl logs -n kube-system kube-scheduler-controlplane | tail -20
```

Note Static pod manifests: changes cause automatic pod restart by kubelet -- no restart needed. Use crictl when API server is down: crictl pods, crictl logs, crictl ps -a.

LAB 26 · TEST IT

Troubleshoot Cluster Components: API Server, Controller Manager, Scheduler, etcd

Test it

Broken API server manifest is fixed; kubectl get nodes works again. journalctl shows the specific error. All kube-system pods are Running.

Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Troubleshoot Nodes: NotReady, kubelet Failure, cgroup Driver Mismatch, Drain

LAB 27

Node troubleshooting

Diagnose a node transition to NotReady by identifying common causes: kubelet service failure, cgroup driver mismatch between kubelet and containerd, and containerd failure. Practice safely draining a node for maintenance.

You'll build

Stop kubelet, observe NotReady, restart. Introduce cgroup mismatch, diagnose with journalctl, fix config.toml, drain and uncordon node.

Key commands: kubectl get · sudo systemctl · journalctl -u · kubectl drain · » After

LAB 27 · COMMANDS

Troubleshoot Nodes: NotReady, kubelet Failure, cgroup Driver Mismatch, Drain

Step 1 -- Check node status baseline

```
kubectl get nodes -o wide
kubectl describe node node01 | grep -A10 Conditions
```

Step 2 -- Scenario: kubelet service stops

```
# On node01:
sudo systemctl stop kubelet
# On control plane (wait 40s):
kubectl get nodes
# node01: NotReady
# Fix (on node01):
sudo systemctl start kubelet && kubectl get nodes
```

LAB 27 · COMMANDS

Troubleshoot Nodes: NotReady, kubelet Failure, cgroup Driver Mismatch, Drain (cont.)

Step 3 -- Scenario: cgroup driver mismatch

```
journalctl -u kubelet --no-pager -n 30
# Look for: 'failed to create containerd task' or 'cgroupDriver'
grep -i cgroup /var/lib/kubelet/config.yaml
grep -i SystemdCgroup /etc/containerd/config.toml
# Fix: ensure both use cgroupDriver: systemd / SystemdCgroup = true
sudo systemctl restart kubelet containerd
```

Step 4 -- Drain node for maintenance

```
kubectl drain node01 --ignore-daemonsets --delete-emptydir-data
kubectl get nodes
# node01: Ready,SchedulingDisabled
# After maintenance:
kubectl uncordon node01
```

Note After 40s without kubelet heartbeat, node controller marks NotReady. After 5 minutes, pods are evicted to healthy nodes. Drain before any maintenance.

LAB 27 · TEST IT

Troubleshoot Nodes: NotReady, kubelet Failure, cgroup Driver Mismatch, Drain

Test it

Stopped kubelet causes NotReady; starting it restores Ready within 1 minute. Drain evicts pods and cordons node; uncordon re-enables scheduling.

Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

TOPIC 5

Application & Log Troubleshooting

Pods, logs and container debugging

Pod Failure States

- ImagePullBackOff: wrong image name, tag or missing imagePullSecret — check Events in kubectl describe pod.
- CrashLoopBackOff: container exits non-zero repeatedly — read kubectl logs <pod> --previous.
- OOMKilled: container exceeded its memory limit — raise resources.limits.memory.
- kubectl get events --sort-by=.lastTimestamp -A gives a cluster-wide timeline — start here for unknown failures.

Pod Failure States

ImagePullBackOff
wrong image name, tag or missing imagePullSecret

→ kubectl describe pod → Events → 'Failed to pull image'. Fix image ref and recreate.

CrashLoopBackOff
container exits non-zero repeatedly

→ kubectl logs <pod> --previous → read the crash output. Fix command / config / entrypoint.

OOMKilled
container exceeded memory limit

→ kubectl describe pod → Reason: OOMKilled → raise resources.limits.memory.

Fast triage: kubectl get events --sort-by=.lastTimestamp -A · kubectl debug (ephemeral container for distroless)

Application Logs: kubectl logs, --previous, Multi-Container, Raw Log Files on Disk

LAB 28 Application logs

Read logs from running containers, crashed containers (--previous), specific containers in multi-container pods, and raw log files on disk at /var/log/pods/. Log retrieval is the first step in every CKA application troubleshooting question.

You'll build

Run noisy pod, use kubectl logs with --tail --timestamps --follow. Create crashloop pod and read with --previous. Locate raw log file on disk.

Key commands: kubectl run · kubectl logs · ls /var/log/pods/ · » --previous

LAB 28 · COMMANDS

Application Logs: kubectl logs, --previous, Multi-Container, Raw Log Files on Disk

Step 1 -- Basic log commands

```
kubectl run noisy --image=busybox --restart=Never -- \
  sh -c 'i=0; while true; do echo "line $i"; i=$((i+1)); sleep 1; done'
kubectl logs noisy --tail=5
kubectl logs noisy --tail=10 --timestamps=true
```

Step 2 -- Crashed container logs with --previous

```
kubectl run crasher --image=busybox --restart=Always -- \
  sh -c 'echo starting; sleep 2; echo crashing; exit 1'
kubectl get pod crasher -w
# Watch CrashLoopBackOff
kubectl logs crasher --previous
```

Step 3 -- Multi-container pod logs

```
kubectl logs multi-pod -c c1
kubectl logs multi-pod -c c2
kubectl logs multi-pod --all-containers
```

LAB 28 · COMMANDS

Application Logs: kubectl logs, --previous, Multi-Container, Raw Log Files on Disk (cont.)

Step 4 -- Raw log files on disk

```
ls /var/log/pods/
find /var/log/pods/ -name '*.log' | head -5
tail -20 /var/log/pods/default_noisy_*/noisy/0.log
```

Note --previous reads the last terminated container instance -- essential for CrashLoopBackOff. Raw log files at /var/log/pods/ persist even if the pod is deleted.

LAB 28 · TEST IT

Application Logs: kubectl logs, --previous, Multi-Container, Raw Log Files on Disk

Test it

kubectl logs noisy shows incrementing lines. kubectl logs crasher --previous shows crash message. Raw log file found under /var/log/pods/default_noisy_*/.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

WRAP-UP

Day 3 done — networking, storage and component troubleshooting.

Tomorrow morning: monitoring, service triage, RBAC/scheduling failures. Then mock exam and assessment.

COURSE SLIDES

Certified Kubernetes Administrator (CKA)

WSQ Course Code: TGS-2025054612

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

04

DAY 4 (½ day)

Monitoring, Service Networking & Troubleshooting

Domain 5 final — Labs 29–31 · Then: 12PM lunch · 1PM mock exam · 3:30PM tea · 3:45PM assessment

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

Minimum 75% attendance required

DAY 4 OBJECTIVES

Recap & Today

- Three days of training done — Cluster Architecture, Workloads, Networking, Storage and Troubleshooting.
- This morning — Domain 5 final: monitoring cluster usage, service networking triage, RBAC & scheduling failures.
- Labs 29–31 complete the 31-lab CKA curriculum.
- From 1:00 PM — mock-exam practice on Killercoda; final written assessment from 3:45 PM.

TOPIC 1

Monitoring Cluster Usage

metrics-server, kubectl top and cluster events

Cluster Metrics

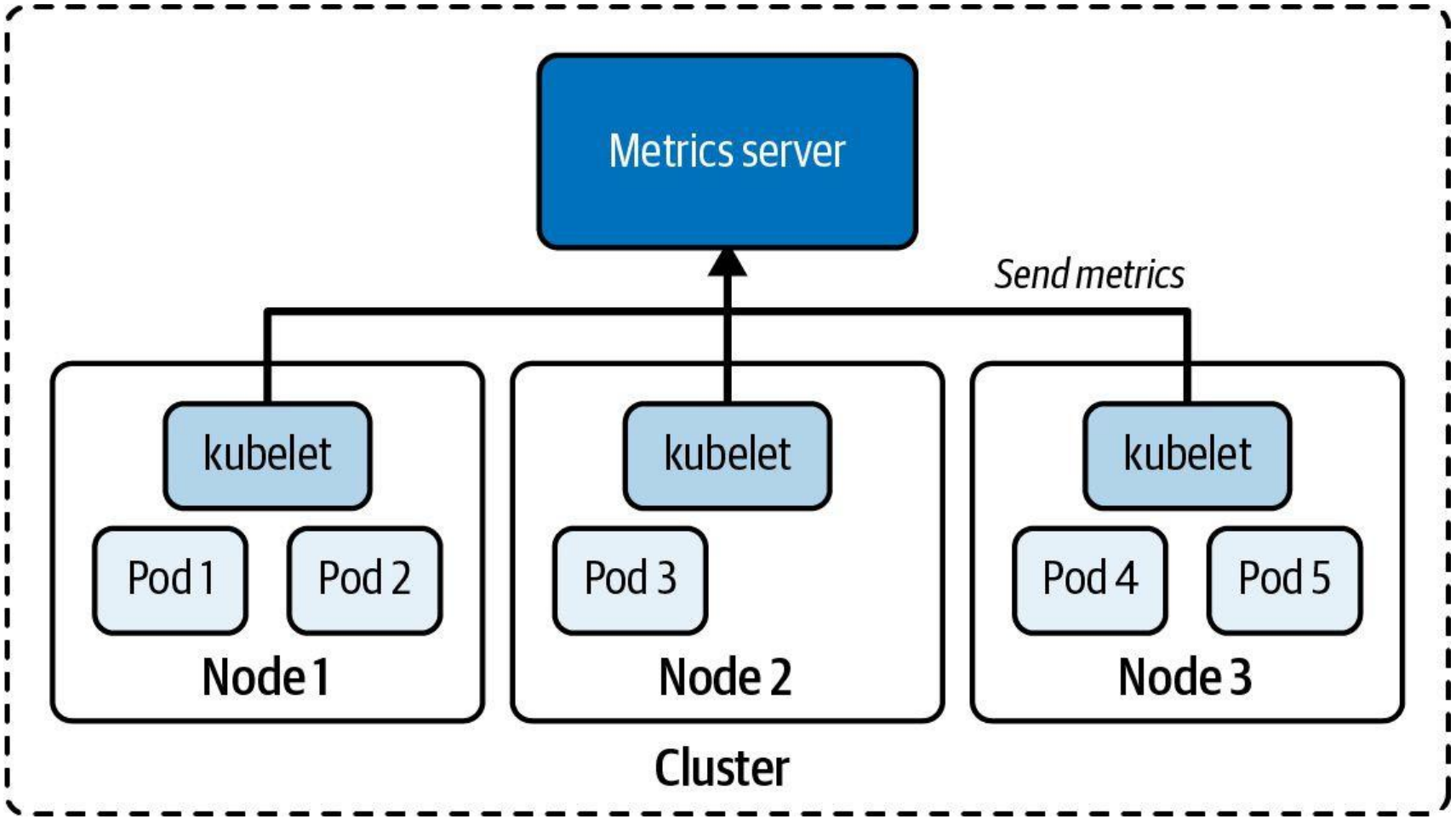
- metrics-server scrapes each kubelet's cAdvisor and exposes the aggregated Metrics API (metrics.k8s.io).
- kubectl top nodes and kubectl top pods show live CPU and memory; --sort-by=cpu/memory ranks results.
- kubectl get events --sort-by=.lastTimestamp -A provides a ~1-hour cluster-wide event timeline.
- kubectl describe node shows capacity vs allocatable vs requested — essential for scheduling failures.

Monitoring — Metrics, Events & Node Health

kubectl top nodes/pods metrics-server → Metrics API (metrics.k8s.io)	Live CPU & memory usage. --sort-by=cpu/memory ranks results. Needs metrics-server running.
kubectl describe node capacity vs allocatable vs requests + Taints + Conditions	First check for scheduling failures — shows resource headroom and all conditions.
kubectl get events -A --sort-by=.lastTimestamp gives ~1h cluster-wide timeline	Find ImagePullErrors, OOMKills, probe failures and rescheduled pods at a glance.
kubectl describe pod Events section — most failure causes appear directly here	Read the Events bottom section first — scheduling reason, pull errors, probe failures all here.

Start every session with **kubectl get events --sort-by=.lastTimestamp -A** — it tells you what happened and when, cluster-wide.

Metrics Server — Cluster-Wide Resource Metrics



DAY 4 · MONITORING

Monitor Cluster and Application Usage: kubectl top, Events, Metrics Pipeline

LAB 29

Monitoring and observability

Use kubectl top to check CPU and memory usage for nodes and pods, inspect cluster events to identify recent warnings, and understand how metrics flow from cAdvisor through metrics-server to kubectl top and HPA.

You'll build

Install metrics-server, run kubectl top node and kubectl top pod (sorted by CPU/memory), inspect kubectl get events, use kubectl describe to surface warning events.

Key commands: kubectl apply · kubectl top · kubectl get · » Metrics

LAB 29 · COMMANDS

Monitor Cluster and Application Usage: kubectl top, Events, Metrics Pipeline

Step 1 -- Install metrics-server

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl -n kube-system patch deploy metrics-server --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
until kubectl top node 2>/dev/null; do echo 'waiting...'; sleep 5; done
```

Step 2 -- Resource usage

```
kubectl top node
kubectl top pod -A
kubectl top pod -A --sort-by=cpu | head -10
kubectl top pod -A --sort-by=memory | head -10
```

Step 3 -- Cluster events

```
kubectl get events -A --sort-by='.lastTimestamp'
kubectl get events -A --field-selector type=Warning
kubectl describe node worker01 | grep -A20 Events
```

LAB 29 · COMMANDS

Monitor Cluster and Application Usage: kubectl top, Events, Metrics Pipeline (cont.)

Step 4 -- Metrics pipeline raw API

```
kubectl get --raw '/apis/metrics.k8s.io/v1beta1/nodes' | python3 -m json.tool | head -30
```

Note Metrics flow: cAdvisor (per node) -> kubelet Summary API -> metrics-server -> kubectl top + HPA. kubectl top requires metrics-server. Events are the first place to look when a pod fails to start.

LAB 29 · TEST IT

Monitor Cluster and Application Usage: kubectl top, Events, Metrics Pipeline

Test it

kubectl top node shows CPU and memory for all nodes. kubectl get events shows Warning events. Metrics API returns JSON with per-node data.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Tea Break

15 minutes

TOPIC 2

Service Networking Triage

Diagnose broken Services, DNS and kube-proxy

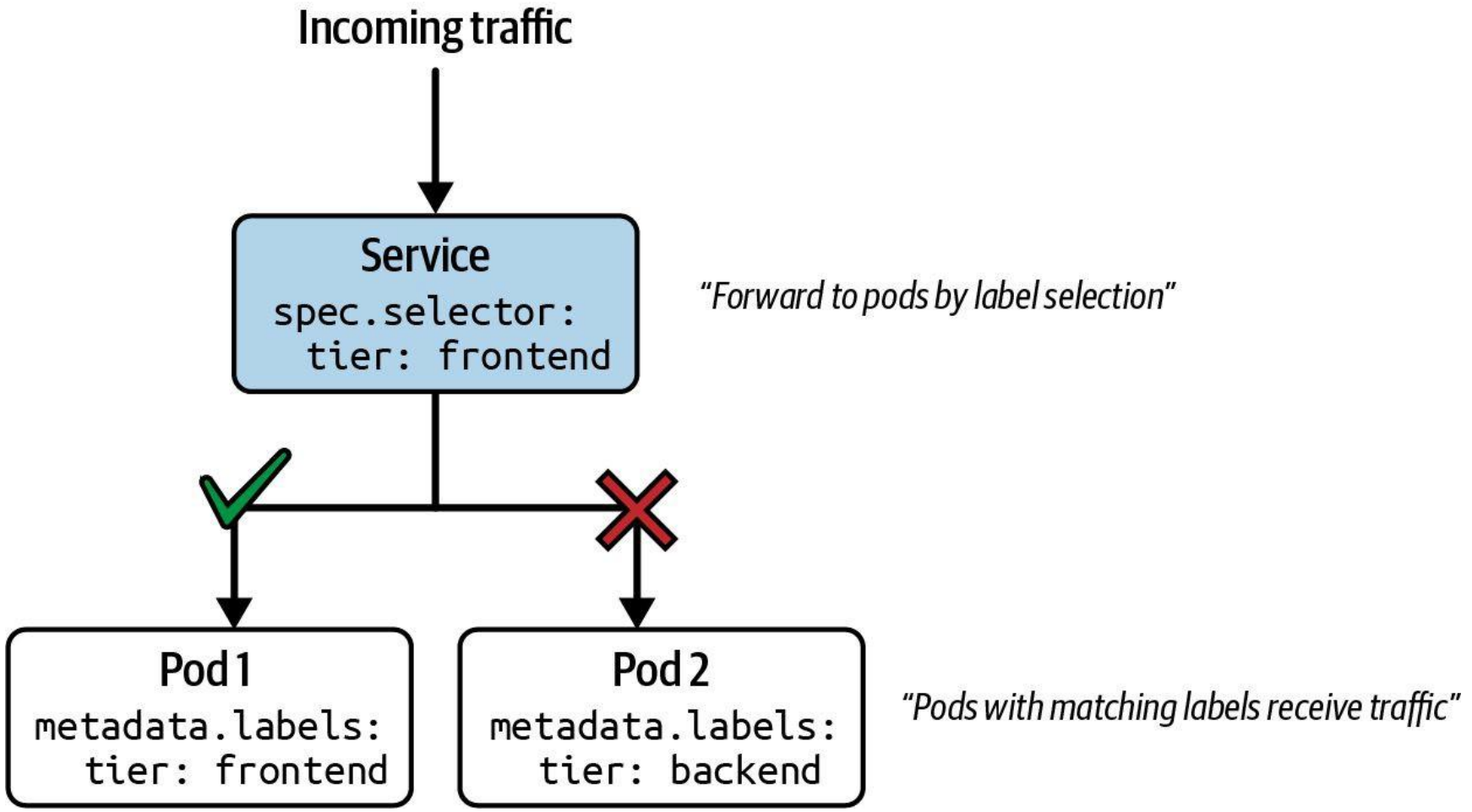
Service Connectivity Debugging

- Empty Endpoints (kubectl get endpoints <svc>) almost always means a selector/label mismatch.
- DNS failure: exec into a pod and run nslookup <service>; check CoreDNS pods are Running.
- kube-proxy programs iptables/IPVS rules — if it crashes, ClusterIP traffic stops routing.
- Bypass the Service to isolate it: curl http://<podIP>:<containerPort> from another pod.

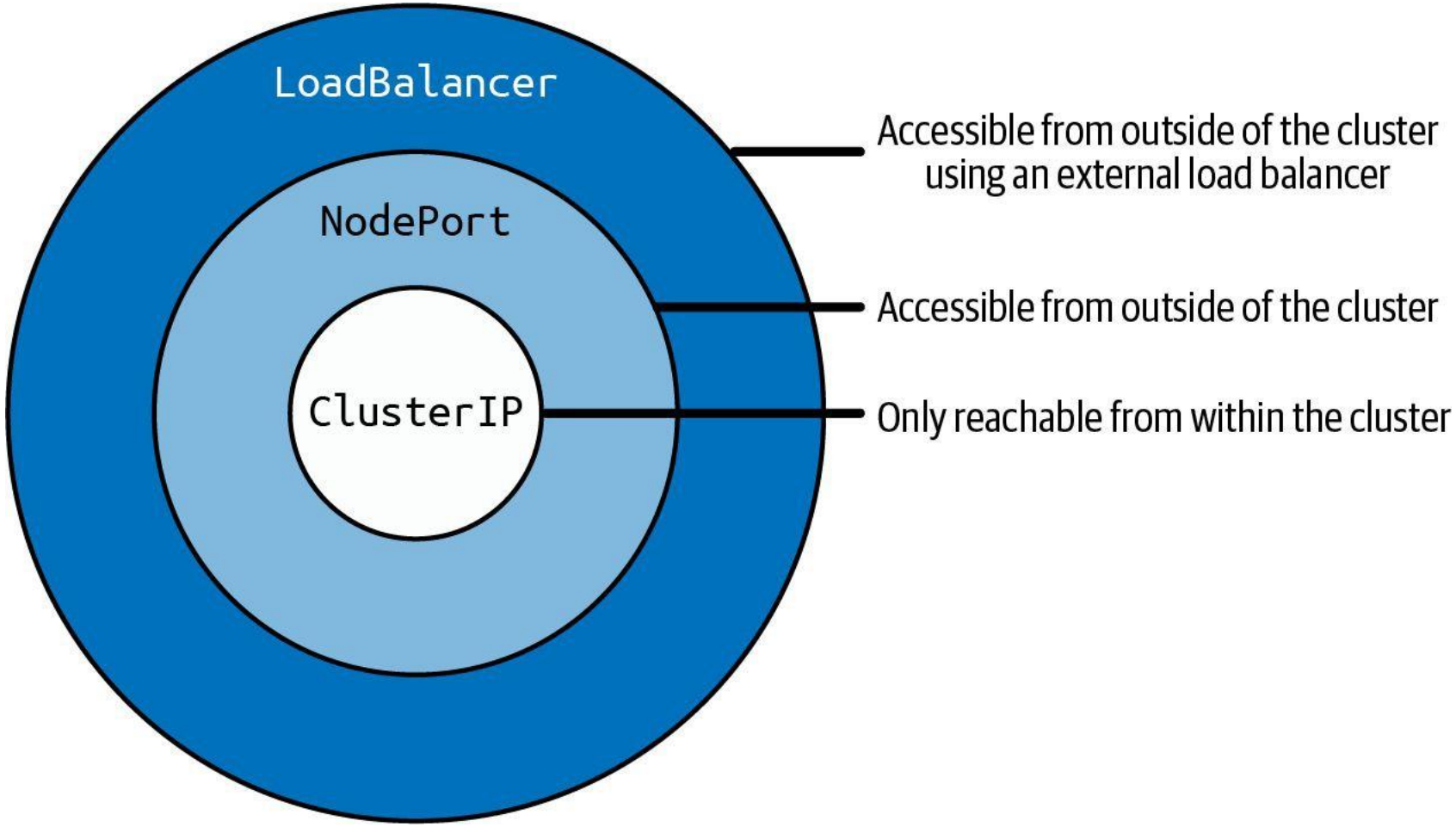
Service Connectivity Triage

DNS resolves?	kubectl exec probe -- nslookup <svc>
Service has endpoints?	kubectl get endpoints <svc> — non-empty?
Selector matches labels?	svc.spec.selector == pod.metadata.labels
targetPort matches?	svc.spec.ports[].targetPort == containerPort
Pod IP reachable?	curl http://<podIP>:<port> — bypasses Service
NetworkPolicy blocking?	kubectl get netpol -n <ns> — check podSelector
kube-proxy alive?	kubectl -n kube-system get pods -l k8s-app=kube-proxy

Service Request Routing — How Traffic Reaches Pods



Service Type Inheritance — ClusterIP to LoadBalancer



DAY 4 · SERVICE NETWORKING

Troubleshoot Networking: DNS, Service Selector, Endpoints, CNI, NetworkPolicy

LAB 30 Network troubleshooting

Follow a structured 7-step triage chain for Kubernetes networking failures: DNS -> Service selector -> Endpoints -> CNI -> NetworkPolicy -> kube-proxy -> routing. CKA 2026 tests this methodical approach in multi-step troubleshooting questions.

You'll build

Deploy baseline app, break DNS by patching CoreDNS, break service selector, block traffic with NetworkPolicy, fix each layer and verify restoration.

Key commands: kubectl create · kubectl exec · kubectl get · » Always

LAB 30 · COMMANDS

Troubleshoot Networking: DNS, Service Selector, Endpoints, CNI, NetworkPolicy

Step 1 -- Deploy baseline

```
kubectl create deployment app --image=nginx --replicas=2
kubectl expose deploy app --port=80 --name=app-svc
kubectl run dbg --image=nicolaka/netshoot --command -- sleep 3600
kubectl exec dbg -- curl -s http://app-svc
```

Step 2 -- Layer 1: DNS check

```
kubectl exec dbg -- nslookup app-svc.default.svc.cluster.local
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl -n kube-system logs -l k8s-app=kube-dns | tail -20
```

Step 3 -- Layer 2: Service and Endpoints

```
kubectl get svc app-svc -o yaml | grep selector -A 5
kubectl get endpoints app-svc
kubectl get pods --show-labels
# Empty endpoints = selector mismatch -- compare and fix
```

LAB 30 · COMMANDS

Troubleshoot Networking: DNS, Service Selector, Endpoints, CNI, NetworkPolicy (cont.)

Step 4 -- Layers 3-7: CNI, NetworkPolicy, kube-proxy

```
kubectl get pods -o wide
# Check pod IPs assigned (CNI OK)
kubectl get networkpolicy -A
# Check for blocking policies
kubectl -n kube-system get pods -l k8s-app=kube-proxy
```

Note Always start with DNS -- 80% of Kubernetes networking problems are DNS. If DNS works but curl fails: check Service selector vs pod labels. If endpoints exist but curl fails: check NetworkPolicy or kube-proxy.

LAB 30 · TEST IT

Troubleshoot Networking: DNS, Service Selector, Endpoints, CNI, NetworkPolicy

Test it

Baseline curl to app-svc works. Selector mismatch: endpoints empty, curl fails. After fixing selector: endpoints repopulated, curl works again.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

TOPIC 3

RBAC & Scheduling Failures

Diagnose 403s, pending pods and node issues

RBAC Troubleshooting

- A 403 Forbidden response means the request was authenticated but not authorised.
- kubectl auth can-i <verb> <resource> --as=<identity> tests permissions without impersonating.
- Check the ServiceAccount's RoleBinding: kubectl get rolebindings,clusterrolebindings -A -o wide.
- Common mistake: Role + ClusterRoleBinding instead of Role + RoleBinding — access is then cluster-wide.

SCHEDULING DEBUG

Scheduling Failures

- Pod stuck in Pending: kubectl describe pod → Events → 'Insufficient cpu/memory' or 'no nodes match'.
- Taint check: kubectl describe node <n> | grep Taint — pod needs a matching toleration.
- nodeSelector / nodeAffinity mismatch: compare spec.nodeSelector with actual node labels.
- Unschedulable node: kubectl get nodes — NotReady or SchedulingDisabled blocks all new pods.

Troubleshoot RBAC (403 Forbidden) and Scheduling Failures (Pending Pod)

LAB 31

RBAC and scheduling troubleshooting

Diagnose and fix two of the most common CKA exam question types: a 403 Forbidden error caused by a missing ClusterRoleBinding, and a pod stuck in Pending due to a scheduling constraint (taint, node selector, or resource exhaustion).

You'll build

Create SA + pod that fails with Forbidden, diagnose with auth can-i, add missing binding. Create pod that is Pending, diagnose with kubectl describe, fix constraint.

Key commands: kubectl create · kubectl run · kubectl describe · » kubectl

LAB 31 · COMMANDS

Troubleshoot RBAC (403 Forbidden) and Scheduling Failures (Pending Pod)

Part A -- RBAC: diagnose 403 Forbidden

```
kubectl create serviceaccount dev-sa
# Pod using dev-sa attempts to list pods:
kubectl exec sa-pod -- kubectl get pods
# Error from server (Forbidden)
kubectl auth can-i list pods --as=system:serviceaccount:default:dev-sa
# no
```

Part A -- Fix: create RoleBinding

```
kubectl create clusterrolebinding dev-sa-view \
  --clusterrole=view \
  --serviceaccount=default:dev-sa
kubectl auth can-i list pods --as=system:serviceaccount:default:dev-sa
# yes
kubectl exec sa-pod -- kubectl get pods
```

LAB 31 · COMMANDS

Troubleshoot RBAC (403 Forbidden) and Scheduling Failures (Pending Pod) (cont.)

Part B -- Scheduling: diagnose Pending pod

```
kubectl run pending-pod --image=nginx
kubectl get pod pending-pod
# Pending
kubectl describe pod pending-pod | grep -A20 Events
# Look for: Insufficient cpu, node(s) had taint, MatchNodeSelector
```

Part B -- Fix scheduling constraints

```
# Taint causing failure:
kubectl describe node worker01 | grep Taints
kubectl taint node worker01 <key>-
# Remove taint
# Or add toleration to pod:
tolerations:
- key: <key>
  operator: Exists
  effect: NoSchedule
kubectl get pod pending-pod -o wide
# Now Running
```

Troubleshoot RBAC (403 Forbidden) and Scheduling Failures (Pending Pod) (cont.)

Note kubectl describe pod is ALWAYS the first command for a Pending pod -- Events shows exactly why. kubectl auth can-i --as is the fastest way to test what a ServiceAccount can do.

LAB 31 · TEST IT

Troubleshoot RBAC (403 Forbidden) and Scheduling Failures (Pending Pod)

Test it

After adding ClusterRoleBinding: SA can list pods, no Forbidden error. After fixing taint/selector: pod transitions from Pending to Running.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

COURSE COMPLETE

All five CKA domains complete.

Architecture · Workloads · Networking · Storage · Troubleshooting.

Lunch Break

1 hour

MOCK EXAM

Mock-Exam Practice — from 1:00 PM

Practise hands-on CKA questions · ask questions freely · killer.sh and Killercoda

WARM-UP

Mock-Exam Practice (1:00 PM)

- Warm-up before the graded assessment — work through CKA-style hands-on tasks.
- Use killer.sh and Killercoda; rehearse alias k=kubectl and export do='--dry-run=client -o yaml'.
- The real CKA exam is 2 hours, 100% hands-on — time yourself on each question.
- Trainer is available for questions; the final assessment begins at 3:45 PM.

Tea Break

15 minutes

ASSESSMENT

Final Assessment — from 3:45 PM

Written / MCQ on the LMS · open book · covers all five CKA domains

FINAL ASSESSMENT

Assessment

Final Assessment

- Written / MCQ assessment on the LMS
- Mock-exam practice from 1:00 PM; final assessment from 3:45 PM
- Covers all five CKA domains and the hands-on labs
- Open book — slides, Learner Guide & kubernetes.io
- Must be attempted for SSG funding

Funding & Competency

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as Competent

Pass the written / MCQ assessment.

ASSESSMENT RULES

Briefing for Assessment

Clear your space

Keep only approved materials on the table.

No recording

No photos of the assessment.

Work individually

No discussion during the assessment.

Use the LMS

The assessment is delivered online.

Open book

Slides, Learner Guide & kubernetes.io are allowed.

Time's up

Submit when time is up.

THANK YOU

Thank you — and good luck with the CKA exam!

Keep practising on killer.sh and Killercoda; the CKA exam is 2 hours, 100% hands-on.